

Trabajo fin de grado

ConectaMayor: aplicación web accesible para conectar a personas mayores con sus familias



Yago Gamo López

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**ConectaMayor: aplicación web accesible para
conectar a personas mayores con sus familias**

**Autor: Yago Gamo López
Tutora: Pilar Rodríguez Marín**

junio 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© Junio de 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, n.º 1
Madrid, 28049
Spain

Yago Gamo López

ConectaMayor: aplicación web accesible para conectar a personas mayores con sus familias

Yago Gamo López

C\ Francisco Tomás y Valiente N.º 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

AGRADECIMIENTOS

A mi tutora, Pilar Rodríguez Marín, por su disponibilidad y por la confianza que ha depositado en este proyecto desde el primer día, así como por la paciencia y la claridad con las que me ha guiado en cada una de las decisiones tomadas a lo largo del desarrollo.

A mi familia, por el apoyo incondicional durante todos estos años de carrera, y de manera muy especial a mis abuelos, que han sido la inspiración de este trabajo y un recordatorio constante de las personas para quienes se diseñaba cada pantalla.

A mis vecinos, que aceptaron participar de manera desinteresada en las pruebas de usabilidad y cuyas observaciones han sido fundamentales para mejorar la aplicación y comprender las necesidades reales de las personas a las que va dirigida.

A mis compañeros de la Escuela Politécnica Superior, con quienes he compartido estos años de trabajo, dudas, exámenes y aprendizaje. Sin ellos, la experiencia universitaria no habría sido la misma.

RESUMEN

La transformación digital de las últimas décadas ha cambiado completamente las formas de comunicación humana, pero también ha aumentado considerablemente la separación con una generación para la que mucha de estas herramientas no fueron pensadas: las personas mayores. El resultado es una brecha digital que, lejos de reducirse, se incrementa debido a la complejidad de las aplicaciones de uso cotidiano y contribuye al aislamiento de un grupo de población que ya presenta cantidades elevadas de soledad no deseada.

Este Trabajo Fin de Grado presenta el diseño, desarrollo y evaluación de *ConectaMayor*, una aplicación web accesible cuyo propósito es facilitar la comunicación entre las personas mayores y sus familiares en un espacio digital privado, sencillo y seguro. La aplicación se organiza en cinco módulos: agenda de recordatorios, listado de contactos, mensajería uno a uno, galería de fotografías compartidas y vinculación familiar mediante código. Esta aplicación contempla tres roles diferenciados, con permisos adaptados al uso que cada miembro de la familia realiza en el sistema: la persona mayor, el familiar editor y el familiar de solo lectura.

El sistema se ha implementado con Django, SQLite, Bootstrap 5 y JavaScript estándar con sondeo AJAX. El diseño de la interfaz se ha guiado por las pautas de accesibilidad WCAG 2.1 en su nivel AA y por los principios del diseño centrado en el usuario, con especial atención a los aspectos visuales como son la tipografía, contraste, áreas de interacción y el uso de un lenguaje sencillo.

La validación se ha llevado a cabo mediante una evaluación de usabilidad con dos personas mayores, siguiendo la metodología propia de la disciplina de Interacción Persona-Ordenador (IPO). Los resultados muestran que la aplicación cumple los objetivos de sencillez y comprensibilidad fijados, además de que han permitido identificar puntos de mejora concretos, incorporados al diseño antes de la entrega final.

PALABRAS CLAVE

Aplicación web, accesibilidad, personas mayores, brecha digital, inclusión digital, Django, WCAG 2.1, usabilidad, diseño centrado en el usuario, interacción persona-ordenador, IPO

ABSTRACT

The digital transformation of recent decades has completely changed the forms of human communication, but has also considerably increased the gap with a generation for whom many of these tools were not designed: older adults. The result is a digital divide that, far from narrowing, keeps growing due to the complexity of everyday applications and contributes to the isolation of a population group that already experiences high levels of unwanted loneliness.

This Bachelor's Thesis presents the design, development and evaluation of *ConectaMayor*, an accessible web application whose purpose is to facilitate communication between older adults and their relatives within a private, simple and secure digital space. The application is organized around five modules: a reminder agenda, a contact list, one-to-one messaging, a shared photo gallery and family linking via code. This application provides three differentiated roles, with permissions adapted to the way each family member uses the system: the older adult, the family editor and the read-only family member.

The system has been implemented using Django, SQLite, Bootstrap 5 and standard JavaScript with AJAX polling. The interface design has been guided by the WCAG 2.1 accessibility guidelines at level AA and by the principles of user-centered design, with particular attention to visual aspects such as typography, contrast, interaction areas and the use of plain language.

Validation has been carried out through a usability evaluation with two older adult participants, following the methodology of the Human-Computer Interaction (HCI) discipline. The results show that the application meets the simplicity and comprehensibility objectives set, and have also made it possible to identify specific improvements that were incorporated into the design before the final submission.

KEYWORDS

Web application, accessibility, elderly people, digital divide, digital inclusion, Django, WCAG 2.1, usability, user-centered design, human-computer interaction, HCI

ÍNDICE

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura del documento	2
2	Estado del arte	5
2.1	Aplicaciones de mensajería generalista	5
2.1.1	WhatsApp	5
2.2	Soluciones específicas para personas mayores	5
2.2.1	GrandPad	6
2.2.2	Mayores Conectados	6
2.3	Plataformas de compartición visual	6
2.3.1	Instagram	6
2.4	Análisis comparativo	7
3	Diseño	9
3.1	Análisis de requisitos	9
3.1.1	Subsistema de gestión de usuarios y vinculación familiar	9
3.1.2	Subsistema de agenda	10
3.1.3	Subsistema de contactos	10
3.1.4	Subsistema de mensajería	11
3.1.5	Subsistema de galería	11
3.1.6	Subsistema de notificaciones	12
3.1.7	Requisitos no funcionales	12
3.2	Arquitectura del sistema	13
3.2.1	Justificación del <i>stack</i> tecnológico	13
3.3	Modelo de datos	15
3.4	Diseño de la interfaz	16
3.4.1	Dos interfaces para un mismo sistema	17
3.4.2	Principios de accesibilidad aplicados	17
3.4.3	Pantallas principales de la aplicación	18

4 Implementación	25
4.1 Tecnologías y entorno de desarrollo	25
4.1.1 Versiones de las tecnologías empleadas	25
4.1.2 Estructura del proyecto	25
4.1.3 Herramientas de desarrollo	26
4.2 Desarrollo por <i>sprints</i>	26
4.2.1 <i>Sprint</i> 0: configuración inicial	27
4.2.2 <i>Sprint</i> 1: autenticación y roles	27
4.2.3 <i>Sprint</i> 2: agenda y contactos	27
4.2.4 <i>Sprint</i> 3: mensajes y galería	28
4.2.5 <i>Sprint</i> 4: iteración del diseño y mejoras	28
4.3 Funcionalidades de la aplicación	29
4.3.1 Autenticación, roles y vinculación familiar	29
4.3.2 Agenda	32
4.3.3 Contactos	32
4.3.4 Mensajería	33
4.3.5 Galería	34
4.3.6 Notificaciones y accesibilidad visual	34
4.4 Evaluación de usabilidad con personas mayores	35
4.4.1 Metodología	35
4.4.2 Perfiles de los participantes	36
4.4.3 Tareas evaluadas	37
4.4.4 Resultados y observaciones	37
4.4.5 Mejoras incorporadas a partir de la evaluación	39
4.4.6 Conclusiones de la evaluación	41
5 Conclusiones	43
5.1 Cumplimiento de los objetivos	43
5.2 Aprendizajes obtenidos	44
5.3 Limitaciones	44
5.4 Trabajo futuro	45
5.5 Reflexión final	46
Bibliografía	49
Apéndices	51
A Pruebas de usabilidad: tablas detalladas	53
A.1 Protocolo de las tareas	53

A.1.1 Tarea T1 — Añadir un recordatorio	53
A.1.2 Tarea T2 — Marcar y modificar un recordatorio	53
A.1.3 Tarea T3 — Llamar a un contacto y dar de alta uno nuevo	54
A.1.4 Tarea T4 — Enviar un mensaje	54
A.1.5 Tarea T5 — Consultar la galería y subir una fotografía	54
A.2 Tablas de incidencias por tarea	54
A.3 Cuestionario final y valoraciones	57
B Entorno y dependencias del proyecto	59
B.1 Entorno de ejecución	59
B.2 Dependencias	59

LISTAS

Lista de códigos

Código 4.1 : Generación automática del código de invitación del grupo familiar.	29
Código 4.2 : Redirección a la pantalla principal según el rol del usuario.	32
Código 4.3 : Sondeo AJAX periódico para la actualización del chat sin recargar la página.	33
Código 4.4 : Variables CSS de los tres temas visuales.	34

Lista de figuras

Figura 3.1 : arquitectura general	13
Figura 3.2 : modelo entidad relación	15
Figura 3.3 : pantallas principales	19
Figura 3.4 : agenda del mayor	20
Figura 3.5 : agenda del editor	21
Figura 3.6 : listado de contactos	22
Figura 3.7 : conversación de mensajes	23
Figura 3.8 : galería y temas	24
Figura 4.1 : pantalla de inicio de sesión	30
Figura 4.2 : formulario de registro	31

Lista de tablas

Tabla 2.1 : Comparativa de aplicaciones	7
Tabla 4.1 : Perfiles de los participantes	36
Tabla 4.2 : Mejoras incorporadas	40
Tabla A.1 : incidencias T1	55
Tabla A.2 : incidencias T2	55
Tabla A.3 : incidencias T3	56
Tabla A.4 : incidencias T4	56
Tabla A.5 : incidencias T5	57
Tabla A.6 : cuestionario final	57

INTRODUCCIÓN

Este primer capítulo presenta el contexto y la motivación del proyecto, define los objetivos que se persiguen y describe la organización del resto de la memoria. El punto de partida es la brecha digital que afecta a las personas mayores. Este problema social guía cada una de las decisiones de diseño, implementación y evaluación desarrolladas en los capítulos siguientes.

1.1. Motivación

Las redes sociales, la mensajería instantánea y las videollamadas han convertido el contacto con familiares y amigos en una cuestión de segundos. Sin embargo, este avance también ha expuesto una brecha entre generaciones. Las personas mayores representan un gran desencuentro entre tecnología y usuario, y es precisamente a ellas a quienes se dirige este proyecto.

El fenómeno, conocido como brecha digital, está ampliamente documentado en España. Los datos del Instituto Nacional de Estadística revelan que una proporción muy significativa de la población mayor de 65 años reside en hogares unipersonales, mientras que los estudios sobre soledad también hablan de grandes números de personas en este rango de edad que han experimentado soledad no deseada [1, 2]. Los estudios sobre las consecuencias de la soledad destacan: deterioro del descanso, aceleración del declive cognitivo y mayor predisposición a enfermedades neurodegenerativas [3]. A ello se suman la distancia respecto a los familiares más cercanos y las limitaciones físicas asociadas al envejecimiento.

La tecnología tiene un potencial evidente para reducir esa distancia, pero solo si se diseña con ese pensamiento. Las plataformas de uso cotidiano, como WhatsApp o las grandes redes sociales, no fueron diseñadas para las personas mayores: incorporan funcionalidades nuevas de forma constante, modifican su interfaz con frecuencia, mezclan acciones distintas en una misma pantalla y emplean iconografía difícil de entender si no estás acostumbrado a ella. Por un lado, muchas personas mayores que se beneficiarían de estar conectadas a la tecnología terminan dependiendo de un familiar para realizar acciones básicas. Por otro, otras renuncian directamente al uso de la tecnología.

ConectaMayor parte de la idea de que es posible construir una herramienta digital sencilla, útil

y privada, capaz de facilitar el contacto cotidiano entre las personas mayores y su entorno familiar. La aplicación no tiene el objetivo de competir con el resto de plataformas. Pero ofrece, en su lugar, un espacio digital sencillo en el que la persona mayor puede gestionar su agenda, consultar sus contactos, intercambiar mensajes con un familiar o ver las fotografías compartidas de la familia.

1.2. Objetivos

El objetivo general de este Trabajo Fin de Grado es **diseñar, desarrollar y evaluar una aplicación web accesible que permita a las personas mayores comunicarse con sus familiares en un espacio digital privado, sencillo y seguro**, aplicando los principios del diseño centrado en el usuario en todas las fases del proyecto.

A partir de este objetivo general, se han establecido los siguientes objetivos específicos:

- 1.— Analizar el contexto del problema y revisar las soluciones existentes desde el punto de vista de la accesibilidad para personas mayores.
- 2.— Diseñar una arquitectura organizada en módulos funcionales: agenda, contactos, mensajería, galería y vinculación familiar. Con un sistema de roles que adapte los permisos y la interfaz al perfil de cada usuario.
- 3.— Implementar una interfaz diferenciada para el rol de persona mayor, con tipografía ampliada, áreas de interacción grandes, lenguaje sencillo y flujos de navegación reducidos al mínimo, manteniendo en paralelo una interfaz completa para los familiares.
- 4.— Cumplir las pautas de accesibilidad WCAG 2.1 en su nivel AA, aplicándolas a colores, tamaños, contrastes, indicadores de foco y mecanismos de interacción.
- 5.— Garantizar la privacidad mediante un modelo de acceso restringido en el que los datos de cada familia permanezcan aislados del resto.
- 6.— Evaluar la usabilidad con personas mayores siguiendo la metodología de la disciplina de Interacción Persona-Ordenador (IPO), e incorporar a la aplicación las mejoras detectadas durante las sesiones.
- 7.— Documentar el proceso, las decisiones técnicas y los resultados de la evaluación.

1.3. Estructura del documento

La memoria se organiza en cinco capítulos y dos apéndices.

El **Capítulo 1** introduce el proyecto. En él se presenta la motivación del trabajo, centrada en la brecha digital que afecta a las personas mayores y en la necesidad de diseñar herramientas tecnológicas que ayuden a su comunicación con el entorno familiar. Además, se definen el objetivo general y los objetivos específicos del proyecto, y se describe la estructura seguida en la memoria.

El **Capítulo 2** presenta el estado del arte. Se analizan las principales aplicaciones existentes que ofrecen soluciones relacionadas con el problema abordado y se discuten sus limitaciones relacionadas con la accesibilidad para personas mayores. Este análisis sitúa *ConectaMayor* en la actualidad y

justifica la idea del proyecto.

El **Capítulo 3** aborda el diseño de la aplicación. Se exponen los requisitos funcionales y no funcionales agrupados por subsistemas, se describe el modelo de datos, se presentan los prototipos de las pantallas principales y se justifican las decisiones de arquitectura más importantes.

El **Capítulo 4** describe la implementación. Se detallan las tecnologías empleadas, la arquitectura elegida, se presentan los módulos desarrollados mediante capturas reales y fragmentos de código, y se incluye, como sección final, la evaluación de usabilidad realizada con personas mayores siguiendo la metodología IPO.

El **Capítulo 5** presenta las conclusiones. Se valora el grado de cumplimiento de los objetivos, se discute las limitaciones encontradas y se proponen ideas y mejoras para el futuro.

El Apéndice A reúne las tablas detalladas de las pruebas de usabilidad, cuyo análisis se ha integrado en el cuerpo del Capítulo 4. El Apéndice B recoge el entorno de ejecución y las instrucciones de instalación necesarias para reproducir el proyecto.

El código fuente completo de la aplicación está disponible públicamente en el repositorio <https://github.com/yagogl/conectamayor-TFG>, donde se incluyen las instrucciones de instalación y ejecución. Los detalles del entorno y las dependencias se recogen en el Apéndice B.

ESTADO DEL ARTE

Este capítulo sitúa *ConectaMayor* en el contexto actual y justifica la idea del proyecto. Para ello se analizan cuatro aplicaciones representativas, con enfoques distintos para abordar la comunicación familiar y la inclusión digital de las personas mayores. El análisis se agrupa en tres bloques: la mensajería generalista, las soluciones específicas para personas mayores y las plataformas de compartición visual. Termina con una comparativa que recoge las conclusiones principales.

2.1. Aplicaciones de mensajería generalista

Las aplicaciones de mensajería generalista son, con diferencia, las más utilizadas por las personas mayores que mantienen contacto digital con su familia, aunque suelen recurrir a ellas por necesidad más que por comodidad.

2.1.1. WhatsApp

WhatsApp [4] es la aplicación de mensajería más extendida en España. Permite enviar mensajes, audios, fotografías y vídeos, hacer llamadas y videollamadas y crear grupos. Esa abundancia de funciones, que para el público general es una ventaja, se convierte en un problema para el usuario mayor. La interfaz reúne en una misma pantalla mensajes individuales, grupos, llamadas, estados, canales y comunidades, sin que el usuario pueda decidir con facilidad qué quiere ver y qué no. WhatsApp favorece la comunicación, pero dentro de un entorno pensado para la población general y poco adaptado al usuario mayor.

2.2. Soluciones específicas para personas mayores

Frente a las plataformas generalistas existe un conjunto reducido de soluciones diseñadas específicamente para personas mayores. Son las referencias más directas para *ConectaMayor*, ya que comparten su punto de partida: reconocer que las herramientas convencionales no cubren bien las

necesidades de este colectivo y proponer un entorno pensado para ellas desde el inicio.

2.2.1. GrandPad

GrandPad [5] es probablemente la solución más conocida a nivel internacional. Es una tableta de hardware específico, con un sistema operativo y aplicaciones diseñadas íntegramente para personas mayores: iconos grandes, un único nivel de navegación y un círculo familiar cerrado en el que solo los allegados autorizados pueden contactar con el usuario. Incluye llamadas, videollamadas, mensajes, fotografías, juegos y un asistente humano accesible con un botón.

El planteamiento es acertado, pero tiene dos limitaciones que lo alejan de muchas familias. La primera es económica, ya que se comercializa por suscripción mensual, un coste difícil de asumir para una parte importante de los hogares. La segunda es de modelo, porque al ser un dispositivo dedicado obliga a adquirir un equipo nuevo en lugar de aprovechar uno que la persona ya tenga.

2.2.2. Mayores Conectados

Mayores Conectados [6] es un proyecto argentino que parte del mismo diagnóstico que este trabajo: fomentar la relación intergeneracional mediante la tecnología, a través de actividades reconocibles para la persona mayor. La plataforma reúne juegos y actividades para que abuelos y nietos las hagan juntos, con interfaces simplificadas.

La diferencia principal con *ConectaMayor* es el enfoque. Mayores Conectados se orienta al ocio compartido, mientras que este proyecto se centra en la comunicación cotidiana, por lo que son enfoques complementarios. Una limitación del proyecto argentino es que parte de sus juegos están alojados en plataformas externas, lo que rompe la sensación de entorno único y puede generar desconfianza al saltar entre dominios.

2.3. Plataformas de compartición visual

El tercer bloque agrupa las aplicaciones cuyo valor principal es compartir contenido visual, sobre todo fotografías. *ConectaMayor* incorpora un módulo de galería con esa misma orientación, por lo que conviene analizar el referente más evidente.

2.3.1. Instagram

Instagram [7] es una de las plataformas de referencia para compartir fotografías, y fue una inspiración directa para el módulo de galería. La idea inicial del proyecto era ofrecer una especie de Instagram

familiar y privado. Su cuadrícula de imágenes con comentarios ha establecido un patrón visual que casi todo el mundo reconoce, incluidas muchas personas mayores que la han visto a través de algún familiar.

Sin embargo, Instagram no encaja en el uso familiar que plantea este proyecto por dos motivos. El primero es su naturaleza abierta, ya que, aunque permite cuentas privadas, está pensada para la difusión pública y la interacción con desconocidos, una superficie de exposición que no encaja con el caso de una familia. El segundo es la saturación de su interfaz, que mezcla la cuadrícula personal con contenido recomendado, mensajería, vídeos cortos y compras. Para una persona mayor, eso convierte algo sencillo como ver las fotos que comparte su familia en una experiencia confusa. *ConectaMayor* retoma la idea visual de Instagram y la lleva a un entorno privado, sin contenido externo y limitado al grupo familiar.

2.4. Análisis comparativo

La Tabla 2.1 resume las características más relevantes de las aplicaciones analizadas frente a *ConectaMayor*. Los criterios elegidos son el diseño específico para mayores, el ámbito privado, la gratuidad, la interfaz unificada y la cobertura funcional, y recogen los aspectos sobre los que se articula el resto de la memoria.

Aplicación	Diseño para mayores	Ámbito privado	Gratuita	Interfaz unificada	Cobertura funcional
WhatsApp	No	Parcial	Sí	No	Media
GrandPad	Sí	Sí	No	Sí	Alta
Mayores Conectados	Sí	Sí	Sí	No	Media
Instagram	No	No	Sí	No	Media
ConectaMayor	Sí	Sí	Sí	Sí	Alta

Tabla 2.1: Comparativa de las aplicaciones analizadas frente a *ConectaMayor*.

Del análisis se desprenden tres conclusiones que justifican el proyecto. La primera es que ninguna de las soluciones gratuitas analizadas combina a la vez un diseño pensado para personas mayores, un ámbito estrictamente privado y una cobertura funcional amplia. WhatsApp es gratuita pero no está adaptada. GrandPad está adaptado y es privado, pero tiene un coste recurrente. Mayores Conectados está adaptado y es gratuito, pero se apoya en plataformas externas. Instagram es gratuito, pero ni está adaptado ni es privado.

La segunda es que las soluciones existentes abordan dimensiones aisladas del problema, como la mensajería, la compartición visual o el ocio, pero no integran un conjunto coherente de servicios en torno a las necesidades cotidianas de la persona mayor. Esa fragmentación obliga a manejar varias aplicaciones para tareas distintas, lo que aumenta la carga cognitiva y la probabilidad de abandono.

La tercera es que ninguna implementa un sistema de roles familiares como el de *ConectaMayor*, en el que un familiar puede asumir la gestión, creando recordatorios, manteniendo los contactos o supervisando el uso, sin invadir el espacio personal de la persona mayor. Este es uno de los rasgos diferenciales del proyecto y se desarrolla en el Capítulo 3.

ConectaMayor no pretende reemplazar a ninguna de estas herramientas, sino ocupar un espacio que hoy no cubre ninguna: una aplicación web gratuita, adaptada a las personas mayores, privada por diseño, con interfaz unificada y con un sistema de roles que reconoce la naturaleza colaborativa del uso familiar de la tecnología.

DISEÑO

Este capítulo recoge las decisiones de diseño adoptadas para *ConectaMayor*. Se parte del análisis de los requisitos del sistema, agrupados por subsistema funcional, y se avanza hacia la arquitectura elegida, el modelo de datos y, por último, el diseño de la interfaz de usuario. El propósito es ofrecer al lector una visión completa del proceso que ha conducido a la solución implementada en el Capítulo 4, justificando las decisiones técnicas más relevantes.

3.1. Análisis de requisitos

El sistema se ha estructurado en seis subsistemas funcionales, cada uno de los cuales agrupa un conjunto coherente de funcionalidades. A esta descomposición se añade un bloque de requisitos no funcionales, que recoge las propiedades transversales del sistema —accesibilidad, privacidad, rendimiento y usabilidad—.

3.1.1. Subsistema de gestión de usuarios y vinculación familiar

Este subsistema gestiona el ciclo de vida de las cuentas de usuario y el mecanismo que permite agruparlas en unidades familiares. Su diseño contempla que un grupo familiar pueda incluir uno o varios usuarios con rol *mayor* —situación habitual cuando conviven matrimonios o hermanos—, así como uno o varios familiares con permisos de gestión o de solo lectura.

RF1.1 El sistema permitirá registrar nuevos usuarios indicando nombre, apellidos, correo electrónico, contraseña y rol.

RF1.2 El sistema permitirá iniciar y cerrar sesión.

RF1.3 El sistema definirá tres roles diferenciados: *mayor*, *familiar editor* y *familiar de solo lectura*.

RF1.4 El sistema redirigirá al usuario, tras iniciar sesión, a una pantalla principal específica para su rol.

RF1.5 Un usuario con rol *familiar editor* podrá crear un grupo familiar, que se identificará mediante un código único generado automáticamente con el formato `FAM-XXXX`.

RF1.6 El resto de roles podrán unirse a un grupo familiar existente introduciendo dicho código en el formulario de registro.

RF1.7 El sistema garantizará que cada usuario pertenezca a un único grupo familiar.

RF1.8 Un grupo familiar podrá contener varios usuarios con rol *mayor*, así como varios familiares con permisos de edición o de solo lectura.

RF1.9 Los usuarios podrán editar la información de su propio perfil. El usuario *familiar editor* podrá editar también la información del perfil de los usuarios *mayores* de su grupo.

3.1.2. Subsistema de agenda

El subsistema de agenda permite registrar y consultar los eventos y recordatorios de cada usuario mayor de manera independiente. Se trata de uno de los módulos con mayor valor práctico, dado que las personas mayores suelen gestionar citas médicas, tomas de medicación y compromisos familiares.

RF2.1 El sistema mostrará a cada usuario *mayor* los eventos programados para los próximos siete días, agrupados por jornada.

RF2.2 Cada evento incluirá un título, una hora, una fecha y un tipo (cita médica, medicación, evento familiar u otros).

RF2.3 El usuario *familiar editor* podrá crear, modificar y eliminar eventos en la agenda de cualquiera de los usuarios *mayores* de su grupo familiar, seleccionando previamente el destinatario.

RF2.4 El usuario *mayor* podrá marcar un evento como completado.

RF2.5 Cada tipo de evento se representará mediante un icono distintivo, de modo que la persona mayor pueda identificarlo sin necesidad de leer el texto completo.

RF2.6 Los eventos estarán asociados exclusivamente al usuario *mayor* al que pertenecen: ningún otro mayor del grupo podrá visualizarlos.

3.1.3. Subsistema de contactos

El subsistema de contactos sustituye, dentro del ámbito de la aplicación, a la agenda telefónica del dispositivo. Al igual que ocurre con la agenda, los contactos pertenecen de forma independiente a cada usuario mayor.

RF3.1 El sistema mostrará los contactos de cada usuario *mayor* agrupados en tres categorías: *familia*, *médicos* y *servicios*.

RF3.2 Cada contacto incluirá nombre, número de teléfono, categoría y, opcionalmente, una nota descriptiva.

RF3.3 Un toque sobre un contacto iniciará automáticamente una llamada telefónica desde el dispositivo del usuario.

RF3.4 El usuario *familiar editor* podrá crear, modificar y eliminar contactos en la lista de cualquiera de los usuarios *mayores* de su grupo familiar, seleccionando previamente el destinatario.

RF3.5 Los contactos estarán asociados exclusivamente al usuario *mayor* al que pertenecen.

3.1.4. Subsistema de mensajería

El subsistema de mensajería habilita la comunicación uno a uno entre los miembros de un grupo familiar. Se ha optado deliberadamente por una mensajería privada y restringida al grupo, sin posibilidad de contactar con personas ajenas a este, como medida de protección frente a fraudes y contactos no solicitados.

RF4.1 Los usuarios podrán enviar mensajes de texto a cualquier otro miembro de su grupo familiar.

RF4.2 La conversación se presentará como un hilo cronológico, diferenciando visualmente los mensajes propios de los recibidos.

RF4.3 El sistema marcará los mensajes como leídos cuando el destinatario abra la conversación.

RF4.4 La interfaz de mensajería actualizará la conversación de forma automática para reflejar los mensajes nuevos sin requerir acción del usuario.

3.1.5. Subsistema de galería

El subsistema de galería permite que los miembros del grupo familiar compartan fotografías en un entorno privado, accesible únicamente para las personas vinculadas al grupo.

RF5.1 Cualquier miembro del grupo familiar podrá subir fotografías a la galería compartida.

RF5.2 La galería se presentará en forma de mosaico, ordenada por fecha de subida descendente.

RF5.3 Cada fotografía se podrá ampliar y consultar en detalle.

RF5.4 Los usuarios podrán dejar comentarios de texto sobre las fotografías.

RF5.5 El usuario que subió una fotografía y el usuario *familiar editor* podrán eliminarla.

RF5.6 El acceso a la galería estará restringido a los miembros del grupo familiar.

3.1.6. Subsistema de notificaciones

El subsistema de notificaciones complementa al resto de módulos mostrando al usuario *mayor*, desde la pantalla principal, la existencia de novedades sin necesidad de acceder a cada sección. Este mecanismo emplea un patrón reconocible para usuarios habituados a otras aplicaciones de uso común y reduce la fricción necesaria para detectar actividad reciente.

RF6.1 La pantalla principal del usuario *mayor* mostrará, junto al icono de cada módulo, un indicador numérico cuando existan novedades pendientes.

RF6.2 El módulo de *agenda* mostrará el número de recordatorios pendientes del día en curso.

RF6.3 El módulo de *mensajes* mostrará el número de mensajes recibidos no leídos.

RF6.4 El módulo de *galería* mostrará el número de fotografías subidas al grupo familiar en las últimas veinticuatro horas.

RF6.5 Los indicadores solo se mostrarán cuando su valor sea mayor que cero, manteniendo la pantalla principal libre de elementos visuales innecesarios cuando no haya novedades.

3.1.7. Requisitos no funcionales

Los requisitos no funcionales recogen las propiedades transversales que debe satisfacer el sistema con independencia de los módulos concretos. Su cumplimiento es especialmente relevante en este proyecto, dado que buena parte de su valor reside en la accesibilidad y la privacidad.

RNF1 Accesibilidad. La interfaz cumplirá las pautas WCAG 2.1 [8] en su nivel AA: contraste mínimo 4,5:1 para texto normal, áreas de interacción de al menos 44 × 44 píxeles, indicadores visibles de foco y compatibilidad con tecnologías de asistencia.

RNF2 Diseño adaptable. La aplicación se visualizará correctamente en pantallas de ordenador, tableta y teléfono móvil sin requerir instalación adicional.

RNF3 Privacidad. Los datos de cada grupo familiar estarán aislados del resto: ningún usuario podrá acceder a información perteneciente a un grupo familiar distinto del suyo.

RNF4 Simplicidad de uso. Las acciones más frecuentes estarán accesibles en un único toque desde la pantalla principal del usuario *mayor*.

RNF5 Rendimiento. El tiempo de respuesta de las operaciones interactivas no superará los dos segundos en condiciones normales de red local.

RNF6 Mantenibilidad. El código fuente seguirá las convenciones de Django, se organizará en aplicaciones independientes por subsistema y dispondrá de pruebas automatizadas para las funcionalidades críticas.

3.2. Arquitectura del sistema

La aplicación sigue una arquitectura cliente–servidor en la que el navegador del usuario actúa como cliente y un servidor Django gestiona la lógica de negocio y el acceso a los datos. Este modelo es el estándar de facto para aplicaciones web, se ajusta a las necesidades del proyecto y permite aprovechar buena parte de las funcionalidades que proporciona el propio *framework*.

Internamente, Django sigue el patrón *Modelo–Vista–Plantilla* (MVT), una variante del clásico MVC en la que las plantillas asumen el papel de la vista tradicional y las vistas se encargan de la lógica de presentación. Esta separación facilita la mantenibilidad del código y permite ofrecer interfaces diferenciadas según el rol de usuario, reutilizando la misma lógica de negocio en el servidor.

La Figura 3.1 muestra el esquema general de la arquitectura, identificando las capas principales y el flujo de datos entre ellas.

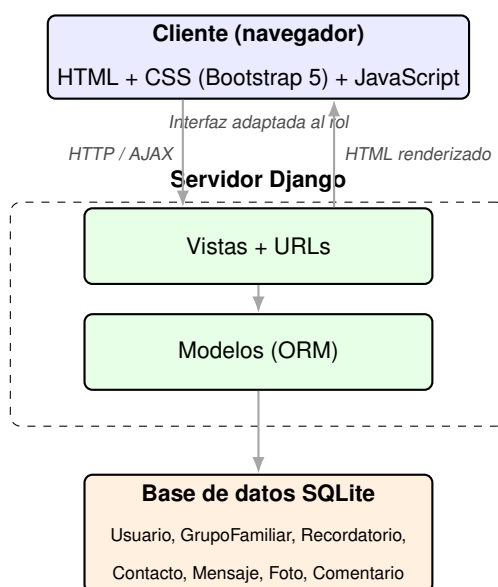


Figura 3.1: Arquitectura cliente–servidor de *ConectaMayor*.

3.2.1. Justificación del *stack* tecnológico

La elección de las tecnologías se ha realizado priorizando la adecuación al proyecto y al perfil del desarrollador frente a la sofisticación técnica. A continuación se justifican las decisiones más relevantes.

Django como *framework* principal.

Django incorpora de serie buena parte de los componentes que cualquier aplicación web necesita: sistema de autenticación, gestión de sesiones, ORM, panel de administración y soporte de internacio-

nalización. Esto reduce el código que es necesario escribir desde cero y permite concentrar el esfuerzo en las funcionalidades específicas de *ConectaMayor*. A ello se suma que Django impone una estructura de proyecto clara, lo que facilita tanto el mantenimiento como la futura ampliación del sistema.

SQLite como sistema de gestión de bases de datos.

SQLite es la opción por defecto de Django y resulta suficiente para el volumen de datos previsto en un entorno familiar: una decena de usuarios por grupo, unos cientos de mensajes y unas pocas docenas de fotografías. Su principal ventaja es que no requiere la instalación de un servidor independiente, lo que simplifica el despliegue y la demostración del sistema. En un escenario de uso real con múltiples familias y mayor concurrencia, la migración a PostgreSQL sería recomendable; se contempla como línea de trabajo futuro.

Bootstrap 5 para la capa de presentación.

Bootstrap aporta un sistema de rejilla responsivo, componentes accesibles por defecto y una base estética profesional sin necesidad de diseñar una interfaz desde cero. Su comportamiento adaptable a distintos tamaños de pantalla es especialmente relevante para una aplicación cuyo público objetivo puede utilizar distintos dispositivos —ordenador, tableta o teléfono—.

JavaScript estándar con sondeo AJAX para la mensajería.

La actualización del chat se ha resuelto mediante peticiones AJAX periódicas al servidor —técnica conocida como *polling*— en lugar de WebSockets. Esta decisión merece una justificación detallada, puesto que existen alternativas más sofisticadas.

WebSockets ofrece comunicación bidireccional en tiempo real con menor latencia, pero introduce una complejidad notable: requiere un servidor ASGI, frente al WSGI estándar de Django, una capa adicional para gestionar las conexiones persistentes y una infraestructura de *routing* que no forma parte del núcleo de Django. Para una aplicación de mensajería familiar, en la que la inmediatez absoluta no es crítica, este coste de complejidad no se compensa con beneficios proporcionales. Un sondeo cada cinco segundos resulta prácticamente equivalente al tiempo real desde la percepción del usuario y permite mantener el proyecto dentro de una pila tecnológica homogénea y manejable.

JavaScript sin librerías de *frontend*.

Se ha descartado el uso de *frameworks* como React o Vue. Las plantillas de Django, combinadas con JavaScript estándar para los aspectos dinámicos puntuales, cubren las necesidades del proyecto sin introducir el coste de aprender y mantener una pila adicional. Esta decisión resulta también prudente dado el nivel de experiencia previa en desarrollo *frontend*, y permite explicar con seguridad cada componente del sistema durante la defensa del trabajo.

3.3. Modelo de datos

El modelo de datos se ha diseñado siguiendo la lógica natural del dominio. El grupo familiar agrupa a varios usuarios, cada uno con su rol correspondiente. Los contenidos de cada módulo se asocian a la entidad que resulta más adecuada en cada caso: los recordatorios y contactos se vinculan al usuario *mayor* concreto al que pertenecen —reflejando que un mismo grupo puede contener varios mayores, cada uno con su propia agenda de recordatorios y su propia lista de contactos—, mientras que la galería se asocia al grupo familiar, ya que su naturaleza es compartida. La Figura 3.2 muestra el diagrama entidad–relación simplificado.

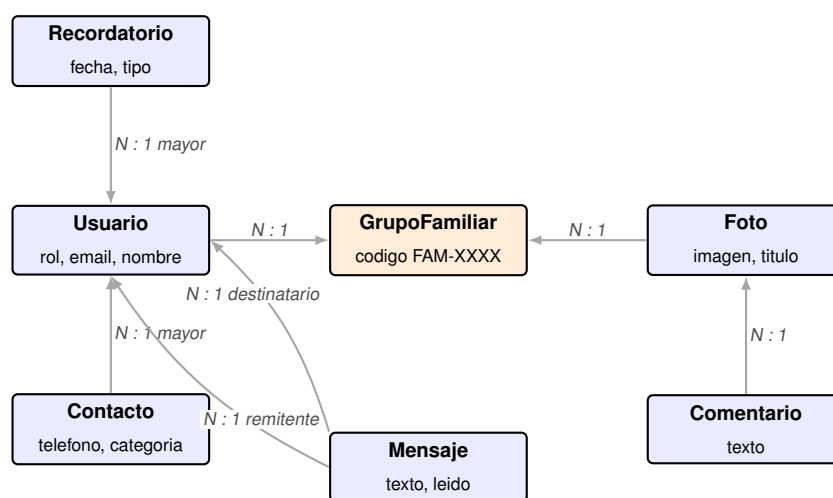


Figura 3.2: Modelo entidad–relación de *ConectaMayor*.

A continuación se describen las entidades principales y los atributos más relevantes de cada una. Los nombres de los modelos se corresponden con los empleados en la implementación.

Usuario.

Extiende el modelo `AbstractUser` de Django añadiendo el rol y el grupo familiar al que pertenece. El rol determina la pantalla principal a la que se redirige al usuario tras iniciar sesión y los permisos que puede ejercer dentro del grupo.

GrupoFamiliar.

Identifica de manera única a una familia dentro del sistema. Se crea automáticamente al registrarse el primer *familiar editor* y genera un código de invitación con el formato `FAM-XXXX`, donde las cuatro últimas posiciones son alfanuméricas aleatorias. Los demás miembros se incorporan al grupo introduciendo este código durante su propio proceso de registro. Un mismo grupo puede contener varios usuarios con rol *mayor*.

Recordatorio.

Cada recordatorio pertenece a un usuario *mayor* concreto, al que está vinculado mediante una relación de clave foránea. Además, almacena el usuario que lo creó —habitualmente un *familiar editor*—, con el objetivo de poder consultar el origen de cada entrada. Incluye fecha, hora y un tipo que determina el icono que se mostrará en la agenda.

Contacto.

De manera análoga al recordatorio, cada contacto pertenece a un usuario *mayor* concreto. Esta vinculación directa con el mayor —y no con el grupo familiar— permite que varios mayores convivan en el mismo grupo manteniendo sus listas de contactos separadas. El contacto incluye un teléfono, una categoría y una nota opcional, y permite iniciar una llamada directa desde la interfaz del usuario mayor.

Mensaje.

Cada mensaje almacena un emisor, un receptor y un texto, junto con la fecha de envío y un indicador de lectura. La conversación entre dos usuarios se reconstruye consultando todos los mensajes en los que ambos participan.

Foto y Comentario.

La fotografía se asocia al grupo familiar y al usuario que la subió, mientras que cada comentario apunta a su fotografía y a su autor. La ordenación de la galería se realiza por fecha descendente, siguiendo el patrón habitual de las redes sociales.

La organización del código en cinco aplicaciones Django independientes —usuarios, agenda, contactos, mensajes y galería— sigue esta misma división, lo que facilita el mantenimiento y la incorporación futura de nuevos módulos sin afectar al resto del sistema.

3.4. Diseño de la interfaz

El diseño de la interfaz es el aspecto en el que *ConectaMayor* se diferencia con mayor claridad de las soluciones generalistas. Las decisiones aquí adoptadas afectan directamente a la experiencia de los usuarios mayores y son las que se evalúan en las pruebas de usabilidad del Capítulo 4.

3.4.1. Dos interfaces para un mismo sistema

La aplicación implementa dos modos de interfaz claramente diferenciados, asociados al rol del usuario:

- La **interfaz simplificada** se presenta al usuario *mayor* y a los *familiares de solo lectura*. Está diseñada para minimizar la carga cognitiva: muestra cuatro accesos directos a los módulos principales, con iconos grandes y etiquetas de texto amplias, y reserva las funciones de gestión para los usuarios con permisos de edición. Junto a cada icono, cuando procede, aparece un indicador numérico con las novedades pendientes, de acuerdo con lo definido en el subsistema de notificaciones.
- La **interfaz completa** se presenta al *familiar editor*. Incluye todos los formularios de creación, edición y borrado de recordatorios y contactos, así como la posibilidad de gestionar el perfil de los usuarios mayores del grupo. En los formularios de agenda y de contactos, cuando el grupo familiar contiene varios mayores, se ofrece un selector previo que permite al editor escoger sobre qué mayor está actuando en cada momento, garantizando así la separación de la información personal de cada uno. Esta interfaz se asemeja, en su densidad informativa, a la de una aplicación de productividad convencional.

El aspecto clave de este diseño es que ambas interfaces se construyen sobre el mismo conjunto de modelos y vistas en el servidor: lo que cambia es la plantilla que se renderiza en cada caso. Esta aproximación, característica de Django, permite mantener una única fuente de verdad para la lógica de negocio y evita la duplicación de código.

3.4.2. Principios de accesibilidad aplicados

El cumplimiento de las pautas WCAG 2.1 en su nivel AA se ha abordado de forma transversal, aplicando los siguientes criterios a todas las pantallas:

- **Tipografía generosa.** El tamaño base del cuerpo de texto es de 18 px, frente a los 14–16 px habituales en aplicaciones generalistas. Los títulos y los textos de los botones principales emplean tamaños proporcionalmente mayores.
- **Contraste cromático.** Todos los textos respetan una relación de contraste mínima de 4,5:1 con respecto a su fondo, y la paleta de colores ha sido validada con la herramienta WebAIM Contrast Checker.
- **Áreas de interacción amplias.** Los botones y enlaces presentan un área activa mínima de 44 × 44 píxeles, en línea con la recomendación de WCAG y con las directrices de las principales guías de diseño táctil.
- **Indicadores de foco visibles.** El contorno de foco del navegador no se elimina mediante CSS y, además, se refuerza con un borde adicional para mejorar su visibilidad en pantallas con poca luz.
- **Lenguaje sencillo.** Los textos de la interfaz evitan tecnicismos y se redactan en imperativo directo (“Guardar”, “Llamar”) en lugar de en formas impersonales.
- **Tres temas visuales.** La aplicación ofrece un tema claro, un tema oscuro y un tema de alto contraste, que el usuario puede alternar desde la barra superior. Esta funcionalidad atiende a usuarios con sensibilidad a la luz o con dificultades visuales específicas.

3.4.3. Pantallas principales de la aplicación

A continuación se muestran capturas reales de las pantallas más representativas de *ConectaMayor*, con la explicación de las decisiones de diseño de cada una. Las imágenes usan los datos de demostración —el grupo González-Pérez, con Carmen y Antonio como usuarios mayores y Laura como familiar editora— y reflejan el aspecto real de la aplicación en uso. Todas se han tomado sobre una tableta en formato vertical, que es el escenario de uso principal previsto. En varias de ellas se ve el botón flotante de *Ayuda*, en la esquina inferior derecha, presente en todas las vistas.

Pantallas principales: mayor y editor.

Las dos pantallas de inicio parten de criterios opuestos. La del rol *mayor* (Figura 3.3, izquierda) reúne lo esencial del día a día sin menús: un saludo personalizado, los recordatorios pendientes de la jornada y, debajo, los cuatro accesos a los módulos en una cuadrícula de iconos grandes. Sobre los iconos con novedades aparece un contador numérico en rojo, según el subsistema de notificaciones de la Sección 3.1.6. La idea es que la persona mayor vea de un vistazo lo que necesita su atención sin explorar nada más.

La pantalla del *familiar editor* (Figura 3.3, derecha) prioriza la densidad de información, porque la maneja alguien más habituado a la tecnología. Destaca el código del grupo familiar (FAM-A3K9), que el editor comparte para que los demás se unan, y una etiqueta junto al nombre que recuerda el rol activo.

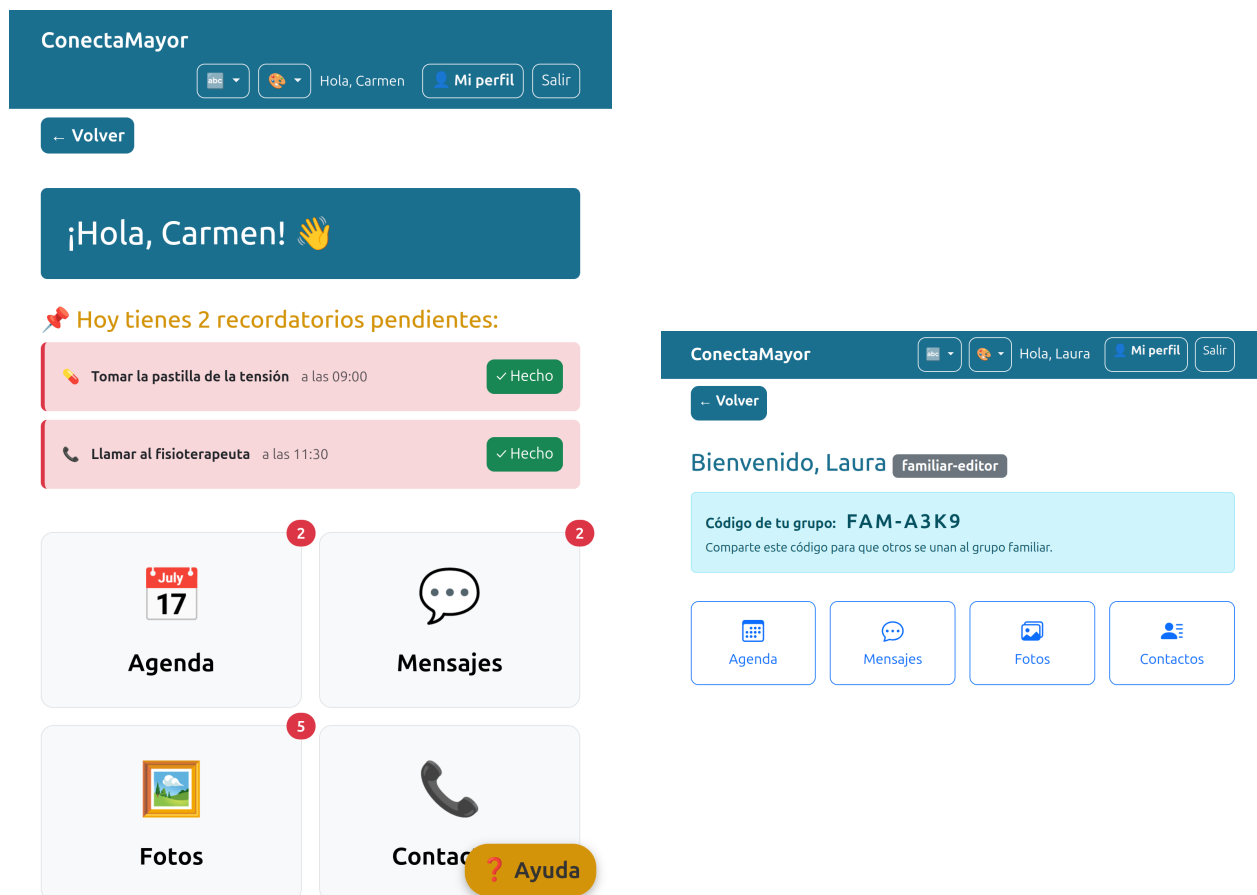


Figura 3.3: Pantallas principales de la aplicación. A la izquierda, el rol *mayor* con los recordatorios del día y los indicadores de novedades; a la derecha, el rol *familiar editor* con el código del grupo destacado.

Agenda del usuario mayor.

La agenda muestra los recordatorios agrupados por días, empezando por el actual. La Figura 3.4 recoge la vista de Carmen. Cada tipo de recordatorio lleva un icono distintivo —una pastilla para la medicación, un teléfono para las llamadas, un hospital para las citas médicas o una tarta para los cumpleaños—, de forma que se reconoce el evento sin leer el título. El botón *Hecho*, verde y grande, marca el recordatorio como completado de un solo toque, en línea con el principio de simplicidad de uso (RNF4).



Figura 3.4: Vista de agenda del usuario mayor, con los recordatorios del día y los siguientes, cada uno con un icono según su tipo.

Agenda del editor: selector de mayor y formulario.

La mayor diferencia entre la interfaz del mayor y la del editor está en la gestión de la agenda. Cuando un grupo tiene varios mayores —como el matrimonio de Carmen y Antonio—, el editor necesita elegir sobre cuál actúa. La Figura 3.5 (izquierda) muestra la solución: un selector en la parte superior con un botón por cada mayor, y el activo resaltado en azul. En la captura, Laura gestiona la agenda de Carmen y puede cambiar a la de Antonio con un clic. Junto a cada recordatorio aparecen, además del botón *Hecho*, los botones de gestión *Cambiar recordatorio* y la papelera para eliminarlo, reservados al editor.

Al pulsar *Añadir recordatorio* se abre el formulario de creación (Figura 3.5, derecha). El encabezado indica para qué mayor se crea el recordatorio (“*Nuevo recordatorio para Carmen Pérez Ruiz*”), sin ambigüedad sobre el destinatario.



Figura 3.5: Gestión de la agenda desde el rol *familiar editor*. A la izquierda, el selector superior para elegir el mayor; a la derecha, el formulario de creación, que indica en su encabezado el destinatario del recordatorio.

Listado de contactos.

La pantalla de contactos sustituye a la agenda telefónica del dispositivo con una organización más accesible. La Figura 3.6 muestra la vista de Carmen. Arriba están los miembros del grupo familiar, identificados con su rol mediante una etiqueta gris. Debajo, los contactos personales agrupados en categorías —*Familia*, *Médico*, *Farmacia*, *Vecino* y *Otro*—. Cada contacto tiene un botón *Llamar* (verde y grande), que inicia la llamada desde el dispositivo, y un botón *Editar* para modificar sus datos.

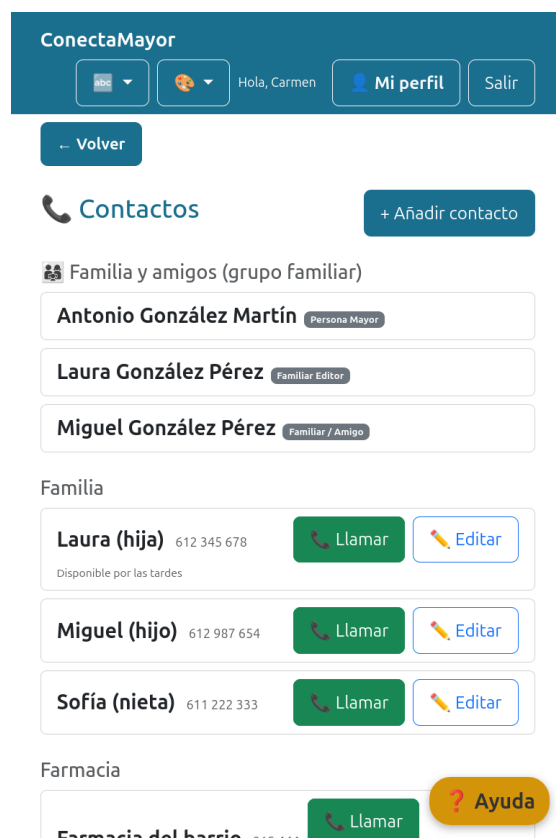


Figura 3.6: Listado de contactos del usuario mayor, agrupados por categorías, con botón de llamada directa.

Conversación de mensajes.

El módulo de mensajería sigue el patrón habitual de las aplicaciones de chat, reconocible para quien haya usado otras antes. La Figura 3.7 recoge una conversación entre Carmen y su hija Laura. Los mensajes recibidos van a la izquierda con fondo claro y los propios a la derecha con el color principal de la aplicación, cada uno con su hora de envío. La conversación se actualiza sola mediante sondeo AJAX, como se justificó en la Sección 3.2.1, sin necesidad de refrescar la página.

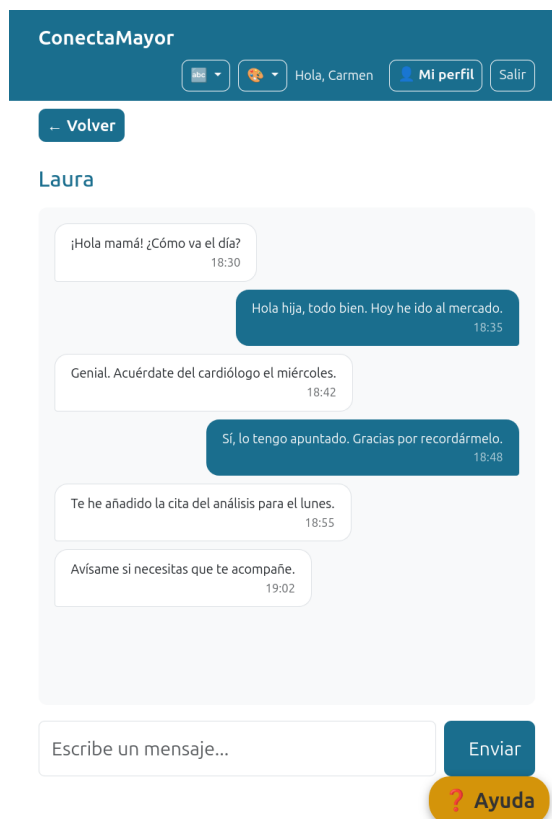


Figura 3.7: Conversación uno a uno, con burbujas diferenciadas para los mensajes propios y los recibidos.

Galería compartida y selector de tema.

La galería presenta las fotografías del grupo en mosaico, ordenadas por fecha de subida descendente (Figura 3.8, izquierda). Las fotos con título y autor muestran esa información bajo la imagen, y el botón *Subir foto* permite a cualquier miembro añadir una nueva.

Como parte del cumplimiento de las pautas WCAG 2.1 nivel AA (Sección 3.4.2), la aplicación ofrece tres temas visuales desde la barra superior (Figura 3.8, derecha). El tema elegido se guarda en el navegador y se mantiene entre sesiones. El de alto contraste se dirige a usuarios con dificultades visuales y el oscuro reduce la fatiga con poca luz.

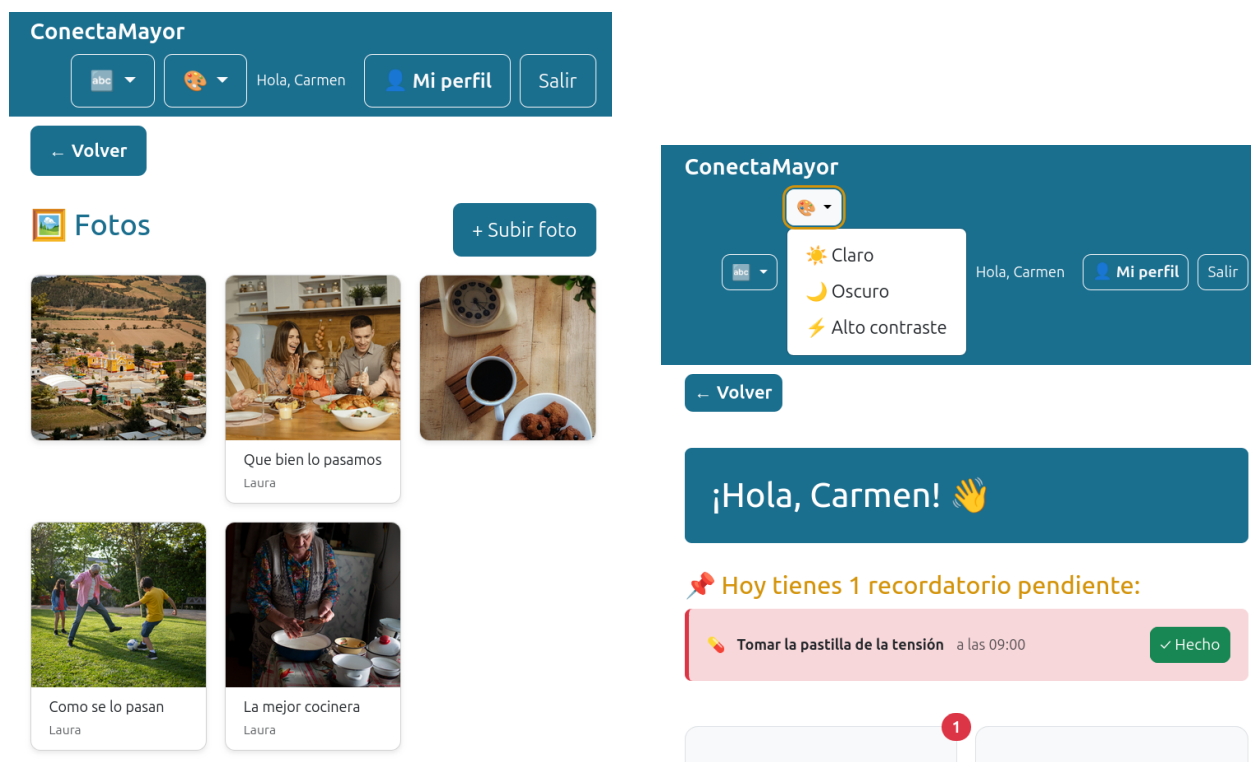


Figura 3.8: A la izquierda, la galería compartida del grupo en formato mosaico; a la derecha, el selector de tema visual desplegado con las tres opciones: claro, oscuro y de alto contraste.

El conjunto de estas decisiones de interfaz da lugar a una experiencia que, como se discute en el Capítulo 4, resultó accesible para los participantes de las pruebas de usabilidad sin que necesitaran formación previa.

IMPLEMENTACIÓN

Este capítulo recoge la materialización del diseño presentado en el Capítulo 3. En primer lugar, se describen las tecnologías empleadas y el entorno de desarrollo utilizado. A continuación, se expone la evolución del proyecto organizada en *sprints*; posteriormente, se presentan las funcionalidades concretas de cada módulo, acompañadas de fragmentos de código representativos; y, por último, se recoge la evaluación de usabilidad realizada con personas mayores siguiendo la metodología propia de la disciplina de Interacción Persona-Ordenador (IPO).

4.1. Tecnologías y entorno de desarrollo

La implementación se ha realizado sobre el *stack* tecnológico justificado en la Sección 3.2.1. En esta sección se detallan las versiones concretas empleadas y la configuración del entorno, con el objetivo de facilitar la reproducibilidad del trabajo.

4.1.1. Versiones de las tecnologías empleadas

El servidor utiliza **Python 3.12.3** sobre **Django 6.0.4**, con la base de datos **SQLite** incluida por defecto en Django. La capa de presentación se apoya en **Bootstrap 5** y en JavaScript estándar (ES2020), sin librerías adicionales. La gestión de dependencias del proyecto se documenta en el fichero `requirements.txt`, lo que permite reproducir el entorno en cualquier sistema compatible mediante un único comando (véase el Apéndice B).

4.1.2. Estructura del proyecto

El proyecto Django se organiza en cinco aplicaciones independientes —una por subsistema funcional—, siguiendo la convención del *framework* y reforzando la separación de responsabilidades discutida en la Sección 3.3. El árbol simplificado de directorios es el siguiente:

```
conectamayor/  
  usuarios/      # Autenticación, roles, grupo familiar  
  agenda/        # Recordatorios  
  contactos/     # Agenda telefónica  
  mensajes/     # Mensajería 1 a 1 con AJAX  
  galeria/       # Fotografías compartidas  
  static/        # CSS, JavaScript, imágenes estáticas  
  templates/     # Plantillas HTML compartidas  
  conectamayor/  # Configuración global del proyecto  
  manage.py  
  requirements.txt
```

Cada aplicación incluye sus propios modelos, vistas, *URLs*, formularios y plantillas. El directorio `templates` de la raíz contiene las plantillas comunes —cabecera, pie de página y estructura base— de las que heredan las plantillas específicas de cada módulo.

4.1.3. Herramientas de desarrollo

El desarrollo se ha llevado a cabo en un equipo con sistema operativo **Ubuntu 24.04 LTS**, utilizando **Visual Studio Code** como editor principal. El control de versiones se ha gestionado con **Git**, con un repositorio local que ha permitido mantener un historial completo de los cambios a lo largo del desarrollo. Cada *sprint* se ha cerrado con un conjunto de *commits* agrupados temáticamente, de forma que el historial refleja la evolución funcional del proyecto.

La ejecución local de la aplicación se realiza con el comando `python manage.py runserver 0.0.0.0:8000`, lo que permite acceder a ella tanto desde el propio equipo como desde otros dispositivos de la red local. Esta posibilidad ha resultado especialmente útil para las pruebas de usabilidad descritas en la Sección 4.4, en las que se accedió al sistema desde una tableta independiente.

4.2. Desarrollo por *sprints*

El proyecto se ha organizado en cinco *sprints* de desarrollo. Esta sección resume cada uno de ellos, identificando los objetivos planteados, las decisiones técnicas más relevantes y las lecciones aprendidas que condicionaron el trabajo posterior.

4.2.1. *Sprint 0: configuración inicial*

El *sprint 0* estuvo dedicado a la preparación del entorno y al diseño preliminar de la arquitectura. Se configuró un entorno virtual con `venv`, se instalaron las dependencias del proyecto y se generó la estructura inicial del proyecto Django. A continuación, se crearon las cinco aplicaciones —`usuarios`, `agenda`, `contactos`, `mensajes` y `galeria`— y se definieron los modelos junto con sus migraciones iniciales.

La decisión más relevante de este *sprint* fue la configuración de `AUTH_USER_MODEL` con un modelo de usuario personalizado antes de ejecutar la primera migración. Aunque Django permite trabajar con su modelo de usuario por defecto, sustituirlo posteriormente es complejo y propenso a errores. Haber tomado esta decisión desde el inicio permitió añadir los campos `rol` y `grupo_familiar` de manera limpia y sin migraciones problemáticas.

Al cierre de este *sprint*, el panel de administración de Django funcionaba correctamente y mostraba los modelos definidos, lo que facilitó la inserción de datos de prueba durante las primeras fases del desarrollo.

4.2.2. *Sprint 1: autenticación y roles*

El *sprint 1* se centró en la implementación del sistema de autenticación, los roles de usuario y el mecanismo de vinculación familiar. Se desarrollaron las vistas de *login*, *logout* y registro, y se establecieron tres roles diferenciados: *mayor*, *familiar editor* y *familiar de solo lectura*. Cada rol redirige al usuario, tras iniciar sesión, a una pantalla principal específica.

El sistema de vinculación familiar mediante código `FAM-XXXX` también se completó durante este *sprint*. La lógica adoptada fue la siguiente: el *familiar editor* crea automáticamente un nuevo grupo familiar al registrarse, mientras que los usuarios *mayor* y *familiar de solo lectura* se incorporan a un grupo existente introduciendo el código de invitación durante su propio proceso de registro.

Durante este *sprint* se habilitaron, además, URL temporales para los módulos pendientes de los *sprints* posteriores. Esto permitió construir un esqueleto navegable del sistema completo desde las primeras fases del desarrollo, facilitando la validación visual del flujo entre pantallas.

4.2.3. *Sprint 2: agenda y contactos*

El *sprint 2* abordó el desarrollo de los módulos de agenda y contactos. En la agenda se implementó la vista de siete días con sus recordatorios, las funcionalidades de creación, edición y borrado, así como la posibilidad de marcar un recordatorio como completado. En el módulo de contactos se implementó el listado por categorías, el botón de llamada directa y las operaciones de gestión equivalentes.

La aportación más significativa de este *sprint* fue la implementación de la **doble interfaz** discutida en el Capítulo 3. Las plantillas se organizaron de modo que cada rol recibiera una versión adaptada a sus necesidades, sin duplicar la lógica de negocio en el servidor: el *familiar editor* accede a formularios completos de gestión, mientras que el usuario *mayor* ve una versión simplificada centrada en la consulta y en las acciones de un único toque.

En esta etapa, los recordatorios y los contactos se asociaban directamente al *grupo familiar*, asumiendo implícitamente que cada grupo contenía un único usuario *mayor*.

4.2.4. *Sprint 3: mensajes y galería*

El *sprint 3* completó los dos módulos restantes del catálogo funcional. La mensajería se resolvió mediante un chat uno a uno, en el que la actualización de la conversación se obtiene mediante *polling* AJAX cada cinco segundos, conforme a la decisión técnica justificada en el Capítulo 3. Los mensajes propios y los recibidos se diferencian visualmente mediante burbujas de distinto color.

La galería se implementó con vista en mosaico, vista en detalle para cada fotografía y soporte para comentarios. La subida de fotos está disponible para todos los miembros del grupo familiar, mientras que el borrado queda restringido al usuario que subió la fotografía y al *familiar editor*. El acceso a la galería está restringido a los miembros del grupo: el sistema filtra las consultas para impedir que un usuario pueda acceder a fotografías de otros grupos familiares.

4.2.5. *Sprint 4: iteración del diseño y mejoras*

Durante el *sprint 4* se incorporaron al sistema dos modificaciones identificadas durante el desarrollo de los *sprints* anteriores, ambas con impacto directo en la experiencia de uso.

En primer lugar, el supuesto inicial de *un grupo familiar equivale a un único usuario mayor* resultó insuficiente en escenarios habituales —por ejemplo, matrimonios o hermanos que conviven— en los que un mismo grupo familiar puede contener a dos o más personas mayores. Esta limitación llevó a refinar el modelo de datos para que las entidades *Recordatorio* y *Contacto* se asociaran al usuario *mayor* concreto en lugar de al grupo familiar, y a introducir en la interfaz del *familiar editor* un selector que permite alternar entre los distintos mayores del grupo. Estos cambios afectan al modelo, a las vistas y a las plantillas de los módulos de agenda y contactos.

En segundo lugar, se incorporó el subsistema de notificaciones descrito en la Sección 3.1.6, consistente en indicadores numéricos que aparecen sobre los iconos de la pantalla principal del usuario mayor cuando existen novedades —recordatorios pendientes del día, mensajes no leídos o fotografías recientes—. Esta mejora replica un patrón visual reconocible para usuarios que hayan tenido contacto con otras aplicaciones de uso común y reduce la fricción necesaria para detectar actividad reciente sin

entrar en cada módulo.

Ambas mejoras se validaron posteriormente en las pruebas de usabilidad descritas en la Sección 4.4, en las que se confirmó su utilidad práctica.

4.3. Funcionalidades de la aplicación

Esta sección describe la implementación de los seis módulos funcionales del sistema. Para cada módulo se resumen las decisiones técnicas más relevantes y, cuando resulta conveniente, se incluye un fragmento de código representativo. El objetivo no es presentar el código en su totalidad, sino mostrar los puntos en los que se materializan las decisiones de diseño discutidas en el Capítulo 3.

4.3.1. Autenticación, roles y vinculación familiar

El módulo de usuarios se ha construido sobre la clase `AbstractUser` de Django, extendida con los campos `rol` —con los tres valores definidos en el catálogo de requisitos— y `grupo_familiar`, que apunta al modelo `GrupoFamiliar` mediante una clave foránea. Esta extensión permite conservar las funcionalidades estándar de Django —autenticación, sesiones y panel de administración— y, al mismo tiempo, incorporar la lógica específica del proyecto.

La pieza más característica de este módulo es la generación automática del código de invitación del grupo familiar. Cada vez que un usuario con rol *familiar editor* se registra, el sistema crea un `GrupoFamiliar` cuyo campo `codigo` se rellena con una secuencia aleatoria con el formato FAM-XXXX. Como muestra el Código 4.1, la generación se delega en una función sencilla que combina letras mayúsculas y dígitos:

```

1 def generar_codigo():
2     """Genera un codigo unico de 6 caracteres tipo FAM-XXXX."""
3     chars = string.ascii_uppercase + string.digits
4     return 'FAM-' + ''.join(random.choices(chars, k=4))
5
6
7 class GrupoFamiliar(models.Model):
8     codigo = models.CharField(
9         max_length=10, unique=True, default=generar_codigo
10    )
11     nombre = models.CharField(max_length=100, blank=True)
12     creado_en = models.DateTimeField(auto_now_add=True)

```

Código 4.1: Generación automática del código de invitación del grupo familiar.

La restricción `unique=True` garantiza que no puedan existir dos grupos con el mismo código. Aunque teóricamente podrían producirse colisiones, el espacio de combinaciones disponible — 36^4 , más de un millón— resulta muy superior al volumen previsto de grupos.

La pantalla de inicio de sesión constituye la puerta de entrada al sistema y se recoge en la Figura 4.1. Su diseño es simple: presenta únicamente los dos campos imprescindibles —usuario y contraseña— y un enlace al formulario de registro para quienes todavía no disponen de cuenta. Al tratarse de la primera interacción del usuario con la aplicación, se ha evitado incluir elementos accesorios que pudieran distraer, sobrecargar la pantalla o generar dudas.

La imagen muestra una interfaz de usuario para la aplicación ConectaMayor. En la parte superior, hay un icono de una casa con un corazón dentro, seguido del título "ConectaMayor" en un color azul. Debajo del título, hay dos campos de entrada: "Usuario" y "Contraseña", ambos con bordes redondeados y un color gris claro. Entre los campos de entrada y el botón "Entrar", hay un espacio vacío. El botón "Entrar" es rectangular, con esquinas redondeadas, de color azul sólido y con el texto "Entrar" en blanco. Debajo del botón, hay un enlace "Regístrate" en azul, precedido por el texto "¿No tienes cuenta?".

Figura 4.1: Pantalla de inicio de sesión, con los campos de usuario y contraseña y el enlace al formulario de registro.

El formulario de registro, mostrado en la Figura 4.2, vincula datos personales básicos, incorpora los dos elementos que articulan el sistema de roles: el selector «¿Quién eres?», mediante el cual el nuevo usuario indica el rol que le corresponde, y el campo del *código del grupo familiar* (FAM-XXXX), que permite a los usuarios mayores y a los familiares de solo lectura incorporarse a un grupo ya existente. El texto de ayuda situado junto al campo de usuario aclara su significado mediante ejemplos sencillos y fáciles de recordar. Esta aclaración constituye una de las mejoras introducidas a raíz de la evaluación de usabilidad descrita en la Sección 4.4.

Crear cuenta

Nombre de usuario (este nombre usarás para entrar en la app)

Elige un nombre fácil de recordar. Por ejemplo: maria, abuela, pepe.

Nombre

Apellidos

Tu número de teléfono (opcional) Para que tu familia pueda llamarte desde la app.

Contraseña

- Su contraseña no puede asemejarse tanto a su otra información personal.
- Su contraseña debe contener al menos 8 caracteres.
- Su contraseña no puede ser una clave utilizada comúnmente.
- Su contraseña no puede ser completamente numérica.

Contraseña (confirmación) Para verificar, introduzca la misma contraseña anterior.

¿Quién eres?

☐ Soy la persona mayor

☐ Soy familiar de confianza (puedo gestionar todo)

☐ Soy familiar o amigo

Nombre del grupo familiar (opcional)

Solo si eres el familiar de confianza que crea el grupo.

Código del grupo familiar Si ya te han dado un código (ej: FAM-TX2K), escríbelo aquí para unirte.

Crear cuenta

[¿Ya tienes cuenta? Inicia sesión](#)

Figura 4.2: Formulario de registro, con el selector de rol y el campo del código del grupo familiar.

La doble interfaz descrita en el Capítulo 3 se materializa en una vista de redirección que dirige a cada usuario, al iniciar sesión, a la pantalla correspondiente según su rol. La vista `inicio` actúa como punto de entrada del sistema y mantiene una lógica deliberadamente sencilla, tal como muestra el Código 4.2:

```
1 def inicio(request):
2     """Redirige al login o a la pantalla correcta segun el rol."""
3     if not request.user.is_authenticated:
4         return redirect('login')
5     if request.user.es_mayor:
6         return redirect('inicio_mayor')
7     return redirect('inicio_familiar')
```

Código 4.2: Redirección a la pantalla principal según el rol del usuario.

A partir de esta redirección, cada rama renderiza una plantilla distinta —`usuarios/mayor/inicio.html` o `usuarios/familiar/inicio.html`— sobre el mismo conjunto de datos de sesión. Este enfoque permite mantener una única fuente de verdad para la lógica de negocio y evita la duplicación de código entre los dos modos de interfaz.

4.3.2. Agenda

El módulo de agenda implementa la gestión de recordatorios para cada usuario *mayor* del grupo familiar. Las vistas principales son la *lista de siete días* —desde el día actual hasta seis días adelante— y los formularios de creación, edición y borrado de recordatorios, accesibles únicamente para el rol *familiar editor*.

La consulta de los recordatorios se construye mediante el ORM de Django, filtrando por usuario y rango de fechas. La interfaz del mayor agrupa los resultados por jornada en la propia plantilla, utilizando el agrupador `regroup` que ofrece el sistema de plantillas. De este modo, se mantiene una presentación clara sin añadir procesamiento innecesario en el servidor.

Como consecuencia del rediseño descrito en el *sprint 4*, la clave foránea principal del modelo `Recordatorio` apunta al usuario mayor concreto (`usuario_mayor`) y, opcionalmente, también al usuario que creó el recordatorio (`creado_por`). Este segundo campo no afecta a la visibilidad del recordatorio, pero permite conservar la trazabilidad de las acciones realizadas por el *familiar editor*.

4.3.3. Contactos

El módulo de contactos sigue una estructura paralela a la del módulo de agenda. Cada contacto se asocia a un usuario *mayor* concreto y se clasifica en una de las cinco categorías predefinidas —*familia*, *médico*, *farmacia*, *vecino* y *otro*—. La presentación por categorías se resuelve, al igual que

en la agenda, en la plantilla mediante el agrupador de Django.

La acción de llamada directa desde la interfaz del mayor se implementa con un enlace HTML que utiliza el esquema `tel:`, delegando la apertura de la aplicación de llamada en el dispositivo del usuario. Esta solución no requiere permisos adicionales ni librerías externas, y funciona de forma coherente tanto en ordenadores con *softphone* configurado como en teléfonos y tabletas con capacidad telefónica.

4.3.4. Mensajería

El módulo de mensajería implementa la comunicación uno a uno entre los miembros del grupo familiar. Los mensajes se almacenan en el modelo `Mensaje`, que recoge el emisor, el receptor, el texto, la fecha de envío y un indicador de lectura. La reconstrucción de una conversación se realiza mediante una consulta que devuelve todos los mensajes en los que participan ambos usuarios, ordenados cronológicamente.

La actualización automática se ha resuelto mediante *polling* AJAX, conforme a la decisión técnica justificada en el Capítulo 3. La página de conversación lanza una petición cada cinco segundos al servidor, que devuelve únicamente los mensajes nuevos en formato JSON. El JavaScript del cliente los inserta dinámicamente al final del hilo y, si la pestaña se encuentra activa, los marca como leídos.

El Código 4.3 recoge el corazón del mecanismo: un `setInterval` que, cada cinco segundos, pide al servidor los mensajes posteriores al último ya visible y los inserta al final del hilo.

```

1 // Polling cada 5 segundos
2 setInterval(() => {
3     fetch(`/mensajes/${otroId}/nuevos/?desde=${ultimoId()}`)
4         .then(r => r.json())
5         .then(data => {
6             data.mensajes.forEach(m => anadirMensaje(m));
7         });
8 }, 5000);

```

Código 4.3: Sondeo AJAX periódico para la actualización del chat sin recargar la página.

La función `ultimoId()` devuelve el identificador del último mensaje renderizado en la página, lo que permite al servidor responder únicamente con los mensajes que falten por mostrar. Esta optimización mantiene el tráfico mínimo aunque la conversación se prolongue durante horas.

Esta aproximación, basada únicamente en tecnologías ya presentes en el proyecto, mantiene el sistema dentro de una pila tecnológica homogénea y ofrece una experiencia suficientemente cercana al tiempo real para el caso de uso planteado.

4.3.5. Galería

El módulo de galería gestiona la subida, consulta y comentario de fotografías compartidas entre los miembros del grupo familiar. La subida se realiza a través de un formulario que utiliza el campo `ImageField` de Django, encargado de almacenar los ficheros en el directorio `media/galeria/` y guardar en la base de datos únicamente la ruta correspondiente. La vista en mosaico aprovecha el sistema de rejilla de Bootstrap, lo que asegura la adaptación a distintas resoluciones sin necesidad de código adicional.

El aislamiento entre grupos familiares se materializa en el filtrado de la *queryset*: la vista selecciona siempre las fotografías pertenecientes al grupo familiar del usuario autenticado. Por consiguiente, ningún usuario puede acceder a fotografías de un grupo familiar distinto del suyo, ni siquiera modificando manualmente las URL.

4.3.6. Notificaciones y accesibilidad visual

Los dos elementos transversales más característicos de la interfaz —las notificaciones de la pantalla principal y los tres temas visuales— se han implementado de forma ligera y reutilizable.

Las notificaciones se calculan en el servidor dentro de la propia vista `inicio_mayor`, mediante tres consultas independientes que cuentan, respectivamente, los recordatorios pendientes del día, los mensajes no leídos y las fotografías subidas en las últimas veinticuatro horas. Las cifras resultantes se pasan a la plantilla y se renderizan únicamente cuando son mayores que cero, manteniendo limpia la pantalla cuando no hay novedades.

Los temas visuales se han resuelto íntegramente con CSS, sin JavaScript adicional más allá del fragmento mínimo encargado de fijar el atributo `data-tema` en el elemento `<html>`. El Código 4.4 muestra la estructura de variables CSS sobre la que se apoyan los tres temas:

```

1  /* Tema claro (predeterminado) */
2  :root {
3      --color-principal: #1a6e8e;
4      --color-acento:    #d4940a;
5      --color-fondo:     #ffffff;
6      --color-texto:     #212529;
7  }
8
9  /* Tema oscuro */
10 [data-tema="oscuro"] {
11     --color-principal: #4fb3d4;
12     --color-acento:    #f6c94e;
13     --color-fondo:     #1a1a2e;
14     --color-texto:     #e8e8e8;
15 }
```

```
16
17 /* Tema de alto contraste */
18 [data-tema="contraste"] {
19     --color-principal: #0000ff;
20     --color-acento:    #ffff00;
21     --color-fondo:     #000000;
22     --color-texto:     #ffffff;
23 }
```

Código 4.4: Variables CSS de los tres temas visuales.

El resto de la hoja de estilos consume estas variables, lo que permite que el cambio de tema sea inmediato y consistente en toda la aplicación sin necesidad de recargar la página. La preferencia del usuario se conserva en el *local storage* del navegador, de modo que se mantiene entre sesiones.

4.4. Evaluación de usabilidad con personas mayores

La evaluación con usuarios constituye uno de los elementos diferenciales de este proyecto y responde a la indicación expresa de la tutora académica. Para llevarla a cabo se ha aplicado una metodología basada en los principios de la Interacción Persona-Ordenador (IPO), centrada en la observación directa y en la verbalización del pensamiento durante la interacción con el sistema.

4.4.1. Metodología

La evaluación ha seguido el método *Think Aloud* —pensamiento en voz alta— mediante observación directa no participante. Esta técnica resulta especialmente adecuada cuando el objetivo es comprender las dificultades que encuentra un usuario mientras interactúa con la aplicación, ya que permite registrar no solo el resultado de cada tarea, sino también el razonamiento del participante y los puntos concretos en los que surgen dudas.

Se realizaron dos sesiones independientes durante la semana del 20 de abril de 2026. Ambas sesiones se desarrollaron en el domicilio del matrimonio participante, en un entorno tranquilo y familiar, con el fin de minimizar el efecto de la presión externa sobre los resultados. Cada sesión tuvo una duración aproximada de treinta minutos y fue conducida únicamente por el autor del trabajo, sin acompañantes adicionales.

Dado el carácter informal de la prueba y la relación de vecindad previa con los participantes, el consentimiento se obtuvo de manera verbal antes de iniciar cada sesión. Se informó a cada participante del propósito de la evaluación, del tratamiento anónimo de los datos recogidos —sin registro de nombres ni datos identificativos, más allá del perfil demográfico básico— y de la posibilidad de interrumpir

la prueba en cualquier momento. En la presente memoria se hace referencia a los participantes como **P1** y **P2**.

El despliegue se realizó en local sobre el ordenador portátil del autor, conectado a la red doméstica del matrimonio. El servidor de Django se ejecutó mediante `python manage.py runserver 0.0.0.0:8000` para hacerlo accesible desde otros dispositivos de la red, y se utilizó una *tablet* Lenovo de aproximadamente diez pulgadas, aportada por el propio autor, como dispositivo de interacción para los participantes. Esta configuración reproduce de forma razonable el escenario de uso previsto para la aplicación.

Las tareas planteadas se diseñaron para cubrir los cuatro módulos principales de la aplicación —agenda, contactos, mensajería y galería— y se complementaron durante la sesión con preguntas exploratorias del evaluador. El objetivo era comprobar no solo si el participante completaba la tarea, sino también su grado de comprensión sobre los formularios subyacentes, las opciones disponibles y la estructura general de la aplicación.

4.4.2. Perfiles de los participantes

Los dos participantes forman un matrimonio jubilado y residen en el mismo domicilio. La selección respondió a un criterio práctico —disponibilidad y consentimiento— y, al mismo tiempo, permitió realizar la prueba sobre un perfil próximo al público objetivo de la aplicación: personas mayores con experiencia tecnológica limitada y con vínculos familiares activos. La Tabla 4.1 recoge sus características más relevantes.

	P1	P2
Sexo	Mujer	Hombre
Edad	67 años	69 años
Estudios	Secundarios	Secundarios
Uso de <i>smartphone</i>	Ocasional	Ocasional
Uso de WhatsApp	Con ayuda	Con ayuda
Uso de ordenador	Nunca	Nunca
Dispositivos habituales	Móvil	Móvil

Tabla 4.1: Perfil de los participantes de las pruebas de usabilidad.

Ambos participantes presentan un perfil tecnológico similar: utilizan con cierta soltura un *móvil* para tareas básicas, pero necesitan ayuda para acciones más elaboradas, como enviar mensajes por aplicaciones de mensajería. Ninguno de los dos ha utilizado ordenador personal con anterioridad. Este perfil encaja con el público objetivo de *ConectaMayor*: usuarios con un manejo limitado de la tecnología que pueden beneficiarse de una interfaz simplificada.

4.4.3. Tareas evaluadas

El protocolo de evaluación se estructuró en cinco tareas que cubren las funciones principales del rol *mayor*. Cada tarea se presentó al participante mediante una instrucción breve y contextualizada, sin ofrecer pistas adicionales sobre los pasos que debía seguir.

T1. Añadir un recordatorio a la agenda. Se pide al participante que registre una cita médica para una fecha futura. Esta tarea evalúa la localización del módulo de agenda, la identificación del botón de creación y la comprensión del formulario, incluyendo la selección de la fecha, el tipo de recordatorio y la hora opcional.

T2. Marcar y modificar un recordatorio. El participante debe localizar el recordatorio del día y marcarlo como completado. A continuación, se le pregunta cómo deshacer la acción y cómo modificar el contenido de un recordatorio existente, explorando así su comprensión del flujo completo de gestión.

T3. Llamar a un contacto y dar de alta uno nuevo. El participante busca el teléfono de su médico de cabecera y inicia una llamada desde la propia aplicación. Posteriormente, se le pide que añada un contacto nuevo, explorando la comprensión del formulario de contactos y el uso de las categorías.

T4. Enviar un mensaje a un familiar. El participante envía un mensaje a un miembro del grupo familiar y lee la respuesta. La tarea evalúa la identificación del módulo de mensajería, la selección del destinatario y la diferenciación visual entre mensajes propios y recibidos.

T5. Consultar la galería y subir una fotografía. El participante consulta el mosaico de fotografías, abre alguna en detalle y deja un comentario. A continuación, se le pide subir una fotografía nueva desde la propia *tablet*, evaluando así tanto la consulta como la aportación de contenido.

4.4.4. Resultados y observaciones

A continuación se sintetizan los resultados de las dos sesiones, agrupados por tarea. Las tablas detalladas con la valoración individual de cada tarea, los tiempos de ejecución y la clasificación de gravedad de cada incidencia se recogen en el Apéndice A.

T1 — Añadir un recordatorio.

Ambos participantes completaron la tarea, aunque con dificultad apreciable. P1 invirtió aproximadamente tres minutos y P2 algo más de cuatro. El principal punto de fricción fue el formulario de creación: en la versión original, el campo de fecha aparecía después del título y del tipo, lo que provocaba dudas sobre si se encontraban en la pantalla correcta. Asimismo, el formulario original no contemplaba un

campo de hora para el recordatorio, lo que resultó extraño a los participantes cuando quisieron asignar una hora concreta a algunas de sus citas; posteriormente se incorporó este campo con carácter opcional, dado que recordatorios habituales como “*tomar la medicación*” o “*llamar a la hija*” no siempre tienen una hora exacta asociada.

P1, además, manifestó al inicio dudas sobre el propio concepto de *nombre de usuario*, identificándolo inicialmente con su nombre real. Esta observación se produjo durante una pregunta exploratoria previa a la tarea, en la que se le consultó cómo crearía su propia cuenta de usuario.

T2 — Marcar y modificar un recordatorio.

La parte central de esta tarea —marcar un recordatorio como completado— fue la más sencilla del conjunto: ambos participantes la resolvieron en un minuto o menos. Sin embargo, la pregunta exploratoria sobre cómo deshacer un recordatorio marcado por error reveló que el botón de deshacer pasaba desapercibido. Del mismo modo, el botón para editar un recordatorio existente se confundía con el de posponerlo al día siguiente, ya que ambos compartían un icono similar y una posición contigua. P1 sugirió durante la sesión diferenciar visualmente los recordatorios pendientes de los completados mediante colores, propuesta que se incorporó en las mejoras posteriores.

T3 — Llamar a un contacto y dar de alta uno nuevo.

La acción de llamar se resolvió sin dificultades en ambas sesiones: los dos participantes comprendieron sin ayuda la agrupación por categorías e identificaron el botón de llamada verde como el elemento que debían pulsar. La dificultad apareció en la segunda parte de la tarea, cuando se les pidió dar de alta un contacto nuevo: el botón de edición de un contacto existente no resultaba evidente, y P1 intentó utilizar el botón *atrás* del navegador para volver a la pantalla anterior, sin obtener el resultado esperado.

T4 — Enviar un mensaje.

Esta tarea fue la que requirió más ayuda. Ambos participantes la completaron, pero con intervención puntual del evaluador. El principal problema detectado fue la ubicación poco visible del campo de escritura del mensaje, situado en la parte inferior de la pantalla y con poca altura visual. P1 señaló también que, en el tema oscuro, las burbujas de conversación perdían contraste, y sugirió aumentar el tamaño de letra del chat.

T5 — Consultar la galería y subir una fotografía.

La consulta de la galería se realizó sin dificultades: ambos participantes identificaron el mosaico de fotografías y abrieron alguna en detalle. La posibilidad de pulsar una fotografía para verla a pantalla completa no se descubrió de forma espontánea y se incorporó posteriormente como mejora. La acción

de subir una fotografía, planteada como extensión de la tarea, resultó confusa: el texto por defecto del selector de archivos —*“Ningún archivo seleccionado”*— fue interpretado por ambos participantes como un mensaje de error, por lo que necesitaron orientación para continuar.

Valoración final.

En la entrevista final, P1 valoró la facilidad de uso general con un 5 sobre 5 y la utilidad percibida con un 4 sobre 5. Identificó como módulos más sencillos los de mensajes, contactos y fotos, y como más complejo el de agenda. A la pregunta sobre si utilizaría la aplicación en su día a día respondió afirmativamente. La valoración global de P2 fue equivalente, con una preferencia ligeramente mayor por el módulo de contactos, en consonancia con su uso habitual del teléfono.

4.4.5. Mejoras incorporadas a partir de la evaluación

Las observaciones recogidas durante las dos sesiones se priorizaron en función de su gravedad y de su coste de implementación. La Tabla 4.2 resume las mejoras aplicadas con posterioridad a las pruebas, todas ellas integradas en la versión actual del sistema.

Mejora aplicada	Tarea afectada
Etiqueta clarificada en el campo <i>usuario</i> del registro	Pre-T1
Campo del código familiar añadido al registro	Pre-T1
Botón <i>Añadir recordatorio</i> en color amarillo acento, más visible	T1
Campo de fecha situado como primer campo del formulario	T1
Campo de hora marcado como opcional	T1
Tarjetas de recordatorio diferenciadas por color (rojo pendiente, verde completado)	T2
Botón <i>Editar recordatorio</i> renombrado y diferenciado del de posponer	T2
Botón <i>atrás</i> global añadido a la barra de navegación	T3
Validación del formato del teléfono mediante expresión regular	T3
Miembros del grupo familiar visibles automáticamente en la lista de contactos	T3
Visualización de fotografías a pantalla completa	T5
Texto del selector de archivos sustituido por uno más claro	T5

Tabla 4.2: Mejoras incorporadas a la aplicación a partir de los hallazgos de las pruebas de usabilidad.

Adicionalmente, durante la reflexión posterior a las sesiones, el autor identificó una segunda limitación: el supuesto inicial de *un grupo familiar equivale a un único usuario mayor* resultaba inadecuado para escenarios como el del propio matrimonio participante, en el que un mismo grupo familiar contiene a dos personas mayores cuyas agendas y contactos deben mantenerse separados. Este hallazgo, aunque no fue verbalizado explícitamente por los participantes durante la prueba, surge directamente del contexto observado y motivó el rediseño del modelo de datos y la introducción del selector de mayor descrito en el Capítulo 3 y en el *sprint* 4 de la Sección 4.2.5.

4.4.6. Conclusiones de la evaluación

Las pruebas con usuarios han confirmado los aciertos principales del diseño y, al mismo tiempo, han evidenciado puntos de mejora que han condicionado el resultado final de la aplicación. La interfaz simplificada del rol *mayor* resultó comprensible para los dos participantes sin necesidad de formación previa, y las tareas de consulta —ver recordatorios, abrir conversaciones y navegar por la galería— se completaron con fluidez. Las dificultades se concentraron, como cabía esperar, en las tareas de creación o modificación, donde los formularios y la disposición de los campos adquieren mayor importancia.

CONCLUSIONES

Este capítulo recoge la valoración global del trabajo realizado. Se revisa el grado de cumplimiento de los objetivos planteados en el Capítulo 1, se exponen los principales aprendizajes obtenidos durante el desarrollo, se reconocen las limitaciones del trabajo presentado y se proponen posibles líneas de continuación.

5.1. Cumplimiento de los objetivos

El objetivo general del proyecto era diseñar, desarrollar y evaluar una aplicación web accesible para facilitar la comunicación entre personas mayores y sus familiares, aplicando los principios del diseño centrado en el usuario a lo largo de todo el proceso. El resultado obtenido —*ConectaMayor*, una aplicación funcional y evaluada con usuarios— permite afirmar que este objetivo se ha cumplido en lo esencial. A continuación se valora el grado de cumplimiento de cada uno de los objetivos específicos:

- 1.— El análisis del estado del arte ha permitido situar *ConectaMayor* en el panorama actual de las soluciones digitales orientadas a personas mayores y justificar las decisiones de diseño adoptadas, especialmente el carácter privado, accesible y acotado de la aplicación frente a las plataformas generalistas.
- 2.— La arquitectura modular se ha implementado en cinco aplicaciones Django independientes —usuarios, agenda, contactos, mensajes y galería— acompañadas del subsistema transversal de notificaciones. El sistema de tres roles diferencia con claridad a la persona mayor, el familiar editor y el familiar de solo lectura, ajustando los permisos y la interfaz a las necesidades de cada perfil.
- 3.— La interfaz simplificada del rol *mayor* se ha materializado en una pantalla principal con cuatro accesos directos a los módulos, tipografía ampliada, indicadores numéricos de novedades y botones de gran tamaño. En paralelo, la interfaz del rol *familiar editor* mantiene las funciones de gestión necesarias sin comprometer la simplicidad de la experiencia diseñada para la persona mayor.
- 4.— El cumplimiento de las pautas WCAG 2.1 nivel AA se ha abordado de forma transversal mediante contraste mínimo 4,5:1, áreas de interacción de al menos 44 × 44 píxeles, indicadores de foco visibles, lenguaje sencillo y soporte de tres temas visuales alternativos.
- 5.— La privacidad se ha garantizado mediante el aislamiento de los grupos familiares en las consultas a la base de datos. Ningún usuario puede acceder a información perteneciente a un grupo familiar distinto del suyo, y la aplicación no integra servicios externos ni publicidad.
- 6.— La evaluación de usabilidad con dos personas mayores, conducida conforme a las recomendaciones de la

Asociación Interacción Persona-Ordenador, ha permitido detectar doce mejoras concretas que se han incorporado a la versión final del sistema. Además, una limitación de diseño identificada durante la reflexión posterior a las pruebas —la asunción inicial de un único usuario mayor por grupo familiar— motivó un rediseño del modelo de datos y de la interfaz del editor.

7.— La presente memoria documenta de forma estructurada el proceso completo: desde el análisis del problema hasta la evaluación con usuarios, incluyendo las decisiones técnicas intermedias y su justificación. Esta documentación sirve tanto como registro del trabajo realizado como base para las posibles ampliaciones futuras descritas más adelante.

5.2. Aprendizajes obtenidos

Más allá del producto entregado, el desarrollo del proyecto ha generado aprendizajes significativos. Entre ellos, destacan especialmente tres.

El primero, de carácter técnico, ha sido la consolidación práctica del *stack* Django en un proyecto de tamaño medio. Aunque durante el grado se han tratado distintos aspectos del desarrollo web, no es lo mismo trabajar con ejemplos guiados que enfrentarse a las decisiones reales de un proyecto iniciado desde cero: estructura de aplicaciones, modelo de usuario personalizado, gestión de migraciones, configuración del servidor y organización del código. La experiencia ha mostrado el valor de tomar ciertas decisiones desde las primeras fases —como la definición de `AUTH_USER_MODEL` antes de la primera migración— y el coste que puede tener posponerlas.

El segundo aprendizaje, de carácter metodológico, ha sido la importancia de evaluar el sistema con usuarios. Las sesiones de usabilidad han revelado problemas que no se manifestaban durante el desarrollo, porque el propio diseñador acaba interiorizando la lógica de su aplicación y deja de percibir algunos puntos de fricción. Observar a una persona mayor utilizar la aplicación durante una sesión de prueba proporciona información cualitativa que ninguna prueba automatizada puede ofrecer. De hecho, varias de las mejoras incorporadas en la versión final habrían pasado desapercibidas sin esta etapa.

El tercer aprendizaje, de carácter personal, ha sido constatar que la accesibilidad no debe entenderse como un añadido cosmético al final del desarrollo, sino como una decisión estructural que condiciona desde la organización de la interfaz hasta la elección de palabras de cada botón. Aplicar las pautas WCAG 2.1 ha exigido revisar componentes que en otro contexto podrían haberse dado por válidos, y ha enseñado a analizar las interfaces digitales con un nivel de atención mayor.

5.3. Limitaciones

El trabajo presentado tiene limitaciones que conviene reconocer explícitamente, tanto por honestidad académica como porque marcan el punto de partida natural de las líneas de continuación.

La primera y más evidente es que la aplicación no se ha desplegado en un entorno de producción real. Todas las pruebas —incluidas las sesiones de evaluación con usuarios— se han realizado sobre un servidor local accesible mediante la red WiFi doméstica del matrimonio participante. Aunque esta configuración ha sido suficiente para validar el comportamiento funcional del sistema, quedan fuera del alcance del proyecto aspectos importantes como el rendimiento bajo concurrencia, la configuración de HTTPS, la administración de un servidor público o la gestión de copias de seguridad.

La segunda limitación afecta a la evaluación de usabilidad. El número de participantes ($n=2$) es reducido para extraer conclusiones cuantitativas generalizables, y ambos participantes comparten un perfil demográfico homogéneo: edades cercanas, mismo nivel formativo e idéntico entorno tecnológico. Aunque los hallazgos son útiles dentro de una evaluación cualitativa orientada a la detección de problemas de uso, una validación más amplia requeriría diversificar los perfiles y aumentar el número de sesiones.

La tercera limitación es técnica. SQLite, elegida como base de datos por su simplicidad y por ser la opción por defecto de Django, resulta adecuada para el desarrollo y para un escenario de demostración, pero no sería la opción más conveniente para un despliegue en producción con múltiples familias accediendo simultáneamente. Asimismo, el conjunto de pruebas automatizadas del proyecto se limita a las funcionalidades críticas y no constituye una *suite* completa, lo que reduce la confianza ante futuras modificaciones del código sin verificación manual.

5.4. Trabajo futuro

A partir de las limitaciones anteriores y de las observaciones recogidas durante la evaluación, se identifican varias líneas naturales de continuación para el proyecto.

Despliegue en producción.

La línea de trabajo futuro más inmediata sería llevar *ConectaMayor* a un servidor accesible públicamente. Esto implicaría migrar la base de datos a PostgreSQL, configurar un servidor de aplicaciones como Unicorn detrás de un *proxy* inverso, habilitar HTTPS mediante certificados de Let's Encrypt y adquirir un dominio. Un despliegue real permitiría ampliar la evaluación con familias diversas, no necesariamente cercanas al autor.

Notificaciones externas.

El subsistema de notificaciones implementado actualmente se limita a indicadores visibles dentro de la propia aplicación. Una ampliación útil sería incorporar notificaciones por correo electrónico —para los familiares— y notificaciones *push* a través del navegador, de modo que los usuarios puedan recibir avisos de recordatorios próximos o mensajes nuevos sin necesidad de tener la aplicación abierta.

Integración con calendarios externos.

Permitir la sincronización del módulo de agenda con servicios como Google Calendar facilitaría que un familiar registrase una cita en su agenda personal y esta apareciese automáticamente como recordatorio en la aplicación del mayor, reduciendo así la duplicación de información entre sistemas.

Evaluación ampliada.

Una nueva ronda de pruebas de usabilidad IPO con un número mayor de participantes y con perfiles más diversos —en edad, formación, autonomía tecnológica y contexto familiar— permitiría comprobar hasta qué punto los hallazgos actuales se mantienen en otros casos de uso y, posiblemente, identificar nuevos puntos de mejora que no han emergido con el matrimonio participante.

Mejoras adicionales de accesibilidad.

La accesibilidad puede llevarse más allá del nivel AA mediante la incorporación de lectura por síntesis de voz para los textos de la interfaz, dictado por voz para los mensajes y recordatorios, y adaptaciones específicas para usuarios con dificultades motrices. La API `SpeechSynthesis` del navegador, ya estandarizada, permitiría abordar la primera de estas mejoras con un esfuerzo de implementación reducido.

5.5. Reflexión final

Este proyecto nació de una motivación personal antes que académica. La idea de *ConectaMayor* no surgió de un enunciado ni de una asignatura, sino de una realidad cercana: observar cómo personas como mis propios abuelos se enfrentan cada día a herramientas digitales que no fueron pensadas para ellas. Esa distancia entre lo que la tecnología promete y lo que realmente ofrece a una persona mayor ha guiado cada decisión del trabajo, desde el tamaño de un botón hasta las palabras escritas en una pantalla.

Las dos sesiones de evaluación con personas mayores han sido una de las experiencias más valiosas del proyecto. Observar cómo una persona interactúa con una interfaz diseñada por uno mismo —y detectar, en tiempo real, los puntos en los que el diseño no resulta tan claro como se esperaba— proporciona un aprendizaje difícil de obtener únicamente mediante documentación técnica o pruebas automatizadas. También ha confirmado que el trabajo tiene sentido más allá del marco académico: las dos personas que participaron en las pruebas valoraron positivamente la idea y manifestaron interés en seguir utilizándola.

Más allá de lo técnico, este Trabajo Fin de Grado ha cambiado mi manera de entender la ingeniería. Durante la carrera es habitual valorar la calidad de un sistema por su eficiencia, su elegancia técnica

o su complejidad. Sin embargo, diseñar para personas mayores obliga a desplazar ese criterio: un sistema es bueno cuando una persona con poca experiencia tecnológica consigue, sin ayuda y sin miedo, hacer aquello que necesita. Esa exigencia —la de ponerse en el lugar de quien va a utilizar lo que uno construye— es probablemente el aprendizaje más duradero que me llevo de este proyecto, y no aparece en ninguna línea de código.

La brecha digital que afecta a las personas mayores no se resolverá con una sola aplicación, pero sí puede reducirse mediante una forma distinta de diseñar tecnología: una forma que sitúe a los usuarios reales —con sus limitaciones, su experiencia y sus expectativas— en el centro del proceso. Si este Trabajo Fin de Grado contribuye, aunque sea modestamente, a reforzar esa perspectiva, su valor no estará únicamente en el código desarrollado, sino también en la manera de entender el diseño tecnológico que ha ayudado a formar.

BIBLIOGRAFÍA

- [1] Instituto Nacional de Estadística, “Encuesta continua de hogares.” https://www.ine.es/prensa/ech_2020.pdf, 2020. Consultado el 12 de diciembre de 2025.
- [2] C. Gala Serra and A. Candela Mollá, “Intervenciones sobre la soledad en el anciano: revisión sistemática.” <https://revistasanitariadeinvestigacion.com/intervenciones-sobre-la-soledad-en-el-anciano-revision-sistemica/>, 12 2021. Revista Sanitaria de Investigación. Consultado el 15 de diciembre de 2025.
- [3] M. A. Parra Rizo, “Cómo afecta la soledad al cerebro de las personas mayores.” <https://theobjective.com/sociedad/2023-12-17/afecta-soledad-cerebro-personas-mayores/>, 12 2023. The Objective. Consultado el 18 de diciembre de 2025.
- [4] WhatsApp LLC, “Whatsapp messenger.” <https://www.whatsapp.com>. Consultado el 20 de enero de 2026.
- [5] GrandPad Inc., “Grandpad: la tableta diseñada para personas mayores.” <https://www.grandpad.net>. Consultado el 28 de enero de 2026.
- [6] J. Scaramuzzo and A. Di Biase, “Mayores conectados.” <https://mayoresconectados.com.ar/nosotros/>. EXO S.A. Consultado el 30 de enero de 2026.
- [7] Meta Platforms, Inc., “Instagram.” <https://www.instagram.com>. Consultado el 5 de febrero de 2026.
- [8] World Wide Web Consortium (W3C), “Web content accessibility guidelines (wcag) 2.1.” W3C Recommendation. <https://www.w3.org/TR/WCAG21/>, 2018. Consultado el 12 de febrero de 2026.
- [9] J. Nielsen, “Severity ratings for usability problems.” Nielsen Norman Group. <https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/>, 1994. Consultado el 22 de abril de 2026.

APÉNDICES

PRUEBAS DE USABILIDAD: TABLAS

DETALLADAS

Este apéndice recoge el material detallado de las dos sesiones de evaluación con personas mayores descritas en la Sección 4.4. Se han trasladado a esta ubicación aquellos elementos que, sin formar parte del análisis principal, permiten verificar el procedimiento seguido y facilitar la reproducción de la evaluación en futuras iteraciones del proyecto.

A.1. Protocolo de las tareas

Cada sesión comenzó con una breve introducción al participante, en la que se explicó el carácter de la prueba, se solicitó el consentimiento verbal y se indicó que podía verbalizar en cualquier momento sus dudas o impresiones. A continuación, se plantearon las cinco tareas en el orden que se recoge a continuación. La redacción literal de las instrucciones se mantuvo lo más natural posible, con el fin de no introducir vocabulario técnico que pudiera condicionar el resultado.

A.1.1. Tarea T1 — Añadir un recordatorio

“Imagine que tiene que ir al médico el viernes que viene a las once de la mañana. Apunte esa cita en la aplicación para que no se le olvide”.

Antes de comenzar la tarea, se preguntó al participante cómo crearía su propio perfil de usuario en la aplicación, con el fin de explorar la comprensión de los conceptos de *nombre de usuario* y *código de grupo familiar*.

A.1.2. Tarea T2 — Marcar y modificar un recordatorio

“Acaba de tomar la pastilla de la tensión que aparece en la pantalla principal. Indique en la aplicación que ya la ha tomado”.

Como ampliación, se preguntó al participante cómo deshacer un recordatorio marcado por error y cómo modificar el título o la hora de uno ya existente.

A.1.3. Tarea T3 — Llamar a un contacto y dar de alta uno nuevo

“Su hija le ha pedido que llame al médico de cabecera. Encuentre su teléfono en la aplicación e inicie la llamada”.

Como ampliación, se pidió al participante que añadiera un contacto nuevo —por ejemplo, el teléfono de un vecino— para evaluar la comprensión del formulario de contactos y de las categorías disponibles.

A.1.4. Tarea T4 — Enviar un mensaje

“Quiere avisar a su hija de que ya ha ido al médico. Envíele un mensaje desde la aplicación y lea la respuesta cuando le llegue”.

A.1.5. Tarea T5 — Consultar la galería y subir una fotografía

“Su hija le ha mandado unas fotos del nieto. Ábralas y déjele un comentario en una de ellas”.

Como ampliación, se pidió al participante que subiera una fotografía propia desde la *tablet*, con el objetivo de comprobar la comprensión del formulario de subida y del selector de archivos.

A.2. Tablas de incidencias por tarea

Para cada tarea se registró, durante la propia sesión, la información que se recoge en las tablas siguientes: la duración aproximada, el resultado —completada, completada con ayuda o no completada— y las incidencias observadas, junto con su gravedad estimada.

La gravedad se clasifica en tres niveles, siguiendo la recomendación de Nielsen para evaluaciones heurísticas [9]: **baja** —el problema causa una molestia menor, pero no impide completar la tarea—, **media** —el problema requiere exploración adicional o produce confusión— y **alta** —el problema impide completar la tarea sin ayuda externa—.

	P1	P2
Duración aproximada	3 minutos	2 – 4 minutos
Resultado	Completada con dificultad	Completada con dificultad
Incidencias	• Confusión sobre el concepto de <i>nombre de usuario</i> (alta). • El campo de fecha no aparecía al principio del formulario (media). • El formulario no incluía un campo de hora para el recordatorio (media). • Botón <i>Añadir recordatorio</i> poco visible (baja).	• Dificultad para localizar el botón de creación (media). • Confusión sobre el orden de los campos del formulario (media). • Echó en falta poder indicar la hora del recordatorio (baja).

Tabla A.1: Resultados e incidencias de la tarea T1 (añadir un recordatorio).

	P1	P2
Duración aproximada	1 minuto	Menos de 2 minutos
Resultado	Completada	Completada
Incidencias	• Confusión entre el botón de posponer y el de editar (media). • Botón <i>Deshacer</i> poco visible (baja). • Recordatorios pendientes y completados no se diferenciaban visualmente (baja).	• Confusión entre el botón de posponer y el de editar (media). • Identificación correcta del botón <i>Hecho</i> (sin incidencia).

Tabla A.2: Resultados e incidencias de la tarea T2 (marcar y modificar un recordatorio).

	P1	P2
Duración aproximada	1 – 2 minutos	1 – 2 minutos
Resultado	Completada	Completada
Incidencias	• Intento de utilizar el botón <i>atrás</i> del navegador, sin obtener el resultado esperado (media). • Botón de edición del contacto poco evidente (baja). • Ausencia de validación del formato del teléfono al dar de alta un contacto (baja).	• Localización correcta del botón <i>Llamar</i> (sin incidencia). • Duda sobre en qué categoría clasificar el contacto nuevo (baja).

Tabla A.3: Resultados e incidencias de la tarea T3 (llamar y dar de alta un contacto).

	P1	P2
Duración aproximada	2 – 4 minutos	2 – 4 minutos
Resultado	Completada con ayuda	Completada con ayuda
Incidencias	• Campo de escritura poco visible, situado en la parte inferior de la pantalla (alta). • En el tema oscuro, las burbujas pierden contraste (media). • Tamaño de letra del chat algo reducido (baja).	• Campo de escritura poco visible (alta). • Identificación correcta de las burbujas propias y recibidas (sin incidencia).

Tabla A.4: Resultados e incidencias de la tarea T4 (enviar un mensaje).

	P1	P2
Duración aproximada	2 – 4 minutos	2 – 4 minutos
Resultado	Completada con ayuda	Completada con ayuda
Incidencias	• No identificó cómo ampliar una foto a pantalla completa (media). • Texto del selector de archivos interpretado como mensaje de error (alta). • Identificación correcta del botón <i>Subir foto</i> (sin incidencia).	• Texto del selector de archivos interpretado como mensaje de error (alta). • Duda sobre si una foto subida era visible para el resto del grupo (baja).

Tabla A.5: Resultados e incidencias de la tarea T5 (consultar galería y subir una foto).

A.3. Cuestionario final y valoraciones

Al finalizar las cinco tareas, se planteó a cada participante un breve cuestionario que combinaba valoraciones cuantitativas —mediante una escala Likert de uno a cinco— con preguntas abiertas sobre la experiencia general. Los resultados se recogen en la Tabla A.6.

Aspecto valorado	P1	P2
Facilidad de uso general (1–5)	5	4
Comprensión de los iconos (1–5)	4	4
Tamaño de los elementos (1–5)	4	4
Utilidad percibida (1–5)	4	5
Confianza al utilizar la aplicación (1–5)	5	4
<i>Preguntas abiertas</i>		
Módulo preferido	Mensajes, contactos y fotos	Contactos
Módulo más difícil	Agenda	Agenda
¿Utilizaría la aplicación?	Sí	Sí

Tabla A.6: Valoraciones del cuestionario final por participante.

Las valoraciones son consistentes entre ambos participantes y coherentes con las incidencias observadas durante las tareas: la mayor dificultad se concentra en la agenda —donde los formularios de

creación son los más complejos del sistema—, mientras que los módulos basados en la consulta o en interacciones directas —contactos, mensajes y galería— recibieron valoraciones más altas. Ambos participantes manifestaron interés en seguir utilizando la aplicación más allá de la sesión de prueba, lo que constituye una validación cualitativa de la utilidad práctica de *ConectaMayor*.

ENTORNO Y DEPENDENCIAS DEL PROYECTO

El código fuente completo de la aplicación se encuentra disponible en un repositorio público, accesible en la siguiente dirección:

```
https://github.com/yagogl/conectamayor-TFG
```

El repositorio incluye, además del código, un fichero `README` con las instrucciones detalladas de instalación y puesta en marcha, así como los datos de acceso de un escenario de demostración que permite explorar la aplicación con contenido de ejemplo.

B.1. Entorno de ejecución

La aplicación se ha desarrollado y probado sobre el siguiente entorno:

- Sistema operativo: Ubuntu 24.04 LTS.
- Lenguaje: Python 3.12.3.
- Framework web: Django 6.0.4.
- Base de datos: SQLite, motor incluido por defecto en Django.
- Capa de presentación: Bootstrap 5 y JavaScript estándar.

B.2. Dependencias

Las dependencias del proyecto se gestionan mediante el fichero `requirements.txt`, que permite reconstruir el entorno en cualquier sistema compatible con una única orden:

```
pip install -r requirements.txt
```

El contenido de dicho fichero es el siguiente:

```
asgiref==3.11.1
Django==6.0.4
pillow==12.2.0
sqlparse==0.5.5
```

El reducido número de dependencias refleja una de las decisiones de diseño del proyecto: minimizar las librerías externas y apoyarse en las herramientas que el propio framework Django proporciona. Las instrucciones completas para clonar el repositorio, instalar las dependencias y ejecutar la aplicación —incluida la carga opcional de los datos de demostración— se encuentran detalladas en el fichero `README` del repositorio.



Universidad Autónoma
de Madrid